Apuntes – MVVM y Repository

Desarrollo de Aplicaciones I · UADE · Guía de Ejercicios: MVVM, Repository y archivos (v1.0)

Repositorio: `MVVMlab` · una rama por ejercicio (`ej01-…` a `ej15-…`) · Felipe Forte · Kotlin + Jetpack Compose

---

Cómo leer estos apuntes. La **Parte 1** resume la teoría (presentación *MVVM con Kotlin y Jetpack Compose*) y los temas más importantes. La **Parte 2** recorre los ejercicios uno por uno: consigna, "En pocas palabras", **"Importante a considerar"**, la explicación con el código real del proyecto, las pruebas en el emulador y las preguntas de la guía. Al final hay **anexos** (organización, cómo ejecutar y probar, preguntas transversales, registro de uso de IA) y el **glosario**.

> **Uso de IA (secciones 1.1 y 19 de la guía)**
> La IA se permite para consultar errores, diagramas, pruebas o fragmentos concretos, sin aceptar propuestas sin revisar. Si se usa para resolver un bloqueo, la guía pide conservar: **problema, fragmento enviado, explicación recibida, cambio realizado y prueba** con que se verificó (plantilla en el Anexo D).

Contenido

1. **Parte 1 · Introducción teórica**
2. **Parte 2 · Ejercicios**
   - Ejercicio 1 — ¿Quién está haciendo demasiado?
   - Ejercicio 2 — Una pantalla no es su estado
   - Ejercicio 3 — Primer ViewModel, sin magia
   - Ejercicio 4 — StateFlow: la pantalla observa
   - Ejercicio 5 — Validar sin esconder la regla en un botón
   - Ejercicio 6 — Repository: cambiar datos sin cambiar la pantalla
   - Ejercicio 7 — Leer un archivo de texto desde el Repository
   - Ejercicio 8 — Guardar texto, cerrar y volver
   - Ejercicio 9 — Cargando, listo o falló
   - Ejercicio 10 — De un texto a una lista persistente
   - Ejercicio 11 — Dependencias: ¿quién crea el Repository?
3. **Anexos**
   - Anexo A · Organización del proyecto en carpetas
   - Anexo B · Cómo ejecutar y probar
   - Anexo C · Preguntas transversales (sección 18 de la guía)
   - Anexo D · Registro breve de uso de IA (sección 19 de la guía)
4. **Glosario**

> **Versión de esta rama**
> Este PDF llega hasta el **Ejercicio 11**. La versión completa está en la rama `main`.

Parte 1 · Introducción teórica

Fuente: presentación de cátedra *MVVM con Kotlin y Jetpack Compose — arquitectura de presentación, estado y programación reactiva*. El punto de partida es un síntoma: una pantalla que obtiene datos, los transforma, valida, guarda y además dibuja. Cada interacción agrega variables y condiciones, y una rotación o un error vuelven difícil saber qué estaba viendo el usuario. **MVVM no nace para agregar clases: busca delimitar responsabilidades y hacer explícito quién es el dueño del estado.**

1. Tres responsabilidades

```
View (Compose) ──intención──► ViewModel ──operación──► Model / datos (Repository)
View (Compose) ◄──estado─── ViewModel ◄────datos──── Model / datos
```

Pieza	Le corresponde	Evitar
View Compose	Dibujar el estado, emitir clics, estado visual local	Consultar la base o archivos desde el composable
ViewModel	Recibir la intención, coordinar, validar, exponer `UiState`	Conocer la Activity o modificar widgets
Repository	Proveer datos (observables) y operaciones de escritura	Saber cómo se dibuja la pantalla

El Repository pertenece a la **capa de datos**; MVVM organiza sobre todo la presentación. Una capa de dominio aparte es opcional.

2. UDF y Observer

- **Unidirectional Data Flow**: los eventos **suben** de la UI al dueño del estado; el estado nuevo **baja** para ser dibujado. No hay varias piezas modificando la misma lista por separado.
- **Observer**: la UI observa un productor (un `StateFlow`) y reacciona cuando cambia; no pregunta todo el tiempo.
- **Programación reactiva**: *UI = f(estado)*. Se modelan cambios observables, no pasos para refrescar controles.

3. Estado y evento

	Estado	Evento
Idea	Qué es verdad ahora	Algo que ocurrió
Ejemplo	“Hay dos notas sin guardar”	“El usuario tocó Guardar”
Quién	Compose lo representa	El ViewModel lo procesa y produce estado nuevo

¿Dónde vive el estado? `remember` (recomposiciones) → `rememberSaveable` (recreación) → **ViewModel** (rotación; *no* sobrevive a la muerte del proceso) → **Repository + persistencia** (más allá de la pantalla). **State hoisting**: el valor y un callback se elevan al ancestro dueño.

4. Kotlin para modelar estado

- `data class` + `copy(...)`: estados inmutables; se publica una versión nueva en vez de mutar una colección compartida.
- `sealed interface`: alternativas excluyentes (*Cargando*, *Disponible*, *Fallo*) que el `when` obliga a cubrir; evita combinaciones imposibles de banderas sueltas. No reemplaza a los campos independientes (búsqueda, filtro).
- Lambdas/callbacks para avisar interacciones; tipos anulables para la ausencia de valor.

5. Corrutinas y Flow

- **Corrutina**: trabajo que puede suspenderse sin bloquear la UI. `suspend fun` marca que puede esperar; `viewModelScope.launch` la inicia y se cancela cuando el ViewModel se destruye. Para disco: `withContext(Dispatchers.IO)`.
- **Flow**: secuencia de valores en el tiempo. **Frío** (arranca al recolectar) vs **caliente** (existe aunque nadie lo mire).
- **StateFlow**: caliente, siempre con valor actual; ideal para el `UiState`. El ViewModel guarda un `MutableStateFlow` privado y expone `asStateFlow()`; actualiza con `_uiState.update { it.copy(...) }`.
- **SharedFlow**: difusión sin valor inicial; una señal puede perderse si nadie observa (no usarla para resultados importantes).
- `collectAsStateWithLifecycle()`: Compose observa el flujo solo mientras la pantalla está activa.
- `stateIn(scope, WhileSubscribed(5_000), inicial)`: convierte el `Flow` del Repository en estado de pantalla.
- Operadores: `map`, `filter`, `combine`, `debounce`, `flatMapLatest`, `catch`; usarlos solo si hay un comportamiento que lo pida.

6. Repository y dependencias

```
interface NotaRepository {                    // contrato chico y estable
    fun observarNotas(): Flow<List<Nota>>
    suspend fun guardar(nota: Nota)
}
class MiViewModel(private val repo: NotaRepository) : ViewModel()   // recibe el contrato, no la
implementación
```

La implementación puede ser memoria, archivo, Room o una falsa para tests: la pantalla no cambia. Quién crea el Repository y se lo da al ViewModel es una decisión aparte (**inyección de dependencias**: un contenedor en el `Application` y una *factory* del ViewModel). Probar el ViewModel = reemplazar el Repository por un *fake* y verificar el estado publicado, sin emulador.

Temas más importantes a considerar

1. **Cada pieza una responsabilidad**: la View dibuja y avisa; el ViewModel decide y expone estado; el Repository guarda y obtiene datos.
2. **UDF**: eventos hacia arriba, estado hacia abajo; una sola fuente de verdad por dato.
3. **Estado inmutable** en un `StateFlow` privado/público con `copy`; la pantalla solo lo observa.
4. **ViewModel ≠ persistencia**: sobrevive a la rotación, no a la muerte del proceso; los datos importantes van al Repository.
5. **Estados de carga con `sealed`**: Cargando / Listo / Error, sin banderas contradictorias.
6. **Disco fuera del hilo principal** (`Dispatchers.IO`) y errores comunicados al usuario, no solo en Logcat.
7. **Depender de contratos** (interfaces) y crear las dependencias en un solo lugar.
8. **Probar el ViewModel sin Android** con repositorios falsos.

Parte 2 · Ejercicios

Cada ejercicio vive en su propia rama y contiene solo su código, organizado por capas (ver Anexo A).

Ejercicio 1 — ¿Quién está haciendo demasiado?

Consigna: tomar una pantalla donde todo está mezclado, marcar dónde se dibuja la interfaz, dónde se validan los datos, dónde se modifica la lista y dónde se guarda, y ubicar cada responsabilidad en *View*, *ViewModel* o *Model/Repository*. Todavía **no** se cambia la implementación.

En pocas palabras

Se analiza una pantalla que hace **todo junta** (dibujar, validar, guardar) y se decide a qué parte de MVVM pertenece cada línea. No se escribe código nuevo: es un ejercicio de *mirar y clasificar*. Es la base de todo lo que sigue.

Importante a considerar

- **"Declarar una variable" no dice nada de la capa.** Hay variables en la View, en el ViewModel y en el Repository. La pregunta útil es: *¿este dato tiene que sobrevivir si cierro la app?*
- **El ViewModel decide QUÉ mostrar; la View decide CÓMO.** "Hay un error" es del ViewModel; "el error va en rojo" es de la View.
- **Una regla de negocio** es algo que valdría igual si la app fuera web ("no se aceptan tareas vacías"). Esas van fuera de la pantalla.
- **Evento ≠ estado.** El click en "Agregar" es un evento (pasó); la lista de tareas es estado (está). Los eventos suben, el estado baja.
- **Ver un `Context` dentro de una pantalla es una pista** de que algo (por ejemplo, guardar archivos) está en el lugar equivocado.

Conceptos base

MVVM = separar en 3 responsabilidades

- `View (Compose)` **dibuja** el estado y **avisa** lo que hace el usuario (clicks, texto). No decide nada.
- `ViewModel` **recibe** la acción del usuario, **decide** (valida, coordina) y **expone** el estado de pantalla.
- `Model / Repository` **obtiene y guarda** los datos (memoria, archivo, Room, API). No sabe nada de pantallas.

UI = f(estado): la pantalla es una función del estado.

¿Cómo decido dónde va cada cosa?

El criterio **no** es "¿declara una variable?" — hay variables en las tres capas. Las preguntas útiles son:

1. **¿Esto dibuja algo?** → View. *(Test: si cambio Compose por una web, ¿tengo que reescribir esta línea? Sí → View.)*
2. **¿Es una regla que valdría igual en cualquier interfaz?** → ViewModel.
3. **¿Este dato tiene que sobrevivir si cierro la app?** Sí → dato real → Repository. No, solo importa mientras uso la pantalla → estado de pantalla → ViewModel.

Ejemplo: en WhatsApp, el mensaje que estás escribiendo (borrador) es de la pantalla; los mensajes ya enviados se guardan.

> **El ViewModel decide QUÉ mostrar. La View decide CÓMO mostrarlo.**
> “Hay un error: tarea vacía” → ViewModel. “El error va en rojo, debajo del botón” → View.

Código analizado (pantalla “mal hecha a propósito”)

```
@Composable
fun PantallaTareas(context: Context, modifier: Modifier = Modifier) {
    var texto by remember { mutableStateOf("") }                    // (A)
    var tareas by remember { mutableStateOf(listOf<String>()) }     // (B)
    var error by remember { mutableStateOf<String?>(null) }         // (C)

    Column(modifier.padding(16.dp)) {                                // (D)
        Text("Mis tareas (${tareas.size})")
        OutlinedTextField(value = texto, onValueChange = { texto = it })

        Button(onClick = {
            if (texto.isBlank()) {                                   // (E)
                error = "La tarea no puede estar vacía"
            } else if (tareas.contains(texto)) {                     // (F)
                error = "Esa tarea ya existe"
            } else {
                tareas = tareas + texto                              // (G)
                File(context.filesDir, "tareas.txt")                 // (H)
                    .writeText(tareas.joinToString("\n"))
                texto = ""                                           // (I)
                error = null
            }
        }) { Text("Agregar") }

        error?.let { Text(it, color = Color.Red) }                   // (J)
        LazyColumn { items(tareas) { Text(it) } }                    // (K)
    }
}
```

Resultado: tabla de responsabilidades (entregable)

Letra	Código	Qué hace	Dónde va
(A)	`var texto`	Borrador que escribe el usuario; no sobrevive al cierre	ViewModel
(B)	`var tareas`	Lista real de tareas; tiene que sobrevivir al cierre	Repository
(C)	`var error`	Mensaje de error actual (dato del estado de pantalla)	ViewModel
(D)	`Column`, `Text`, `OutlinedTextField`	Dibuja la pantalla	View
(E)	`texto.isBlank()`	Regla: no se agregan tareas vacías	ViewModel
(F)	`tareas.contains(texto)`	Regla: no se agregan tareas repetidas	ViewModel
(G)	`tareas = tareas + texto`	Agrega la tarea a la lista (el dueño del dato la modifica)	Repository
(H)	`File(...).writeText(...)`	Guarda las tareas en un archivo	Repository
(I)	`texto = ""`	Limpia el campo solo si la tarea se agregó bien (decisión sobre un dato del ViewModel)	ViewModel
(J)	`error?.let { Text(it, Color.Red) }`	Muestra el error en rojo (el error es del VM; dibujarlo es de la View)	View
(K)	`LazyColumn`	Muestra la lista en pantalla	View

Errores frecuentes (los que cometí y corregí)

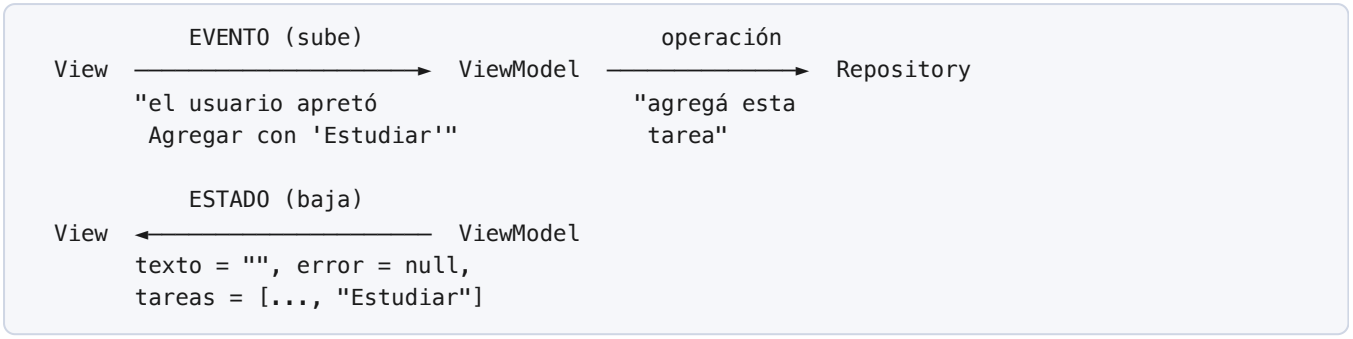
- Pensar que (A), (B) y (C) van al Repository "porque declaran variables". Solo (B) es un dato que debe persistir.
- Poner (I) en la View: `texto` es del ViewModel, y quien es dueño de un dato es quien lo modifica.
- Poner (J) en el ViewModel: el *error* es del ViewModel, pero *mostrarlo* con `Text` y `Color.Red` es de la View.

La cadena completa de "Agregar"

1. View el usuario aprieta *Agregar* → avisa al ViewModel.
2. ViewModel aplica las reglas (E) y (F). Si pasan → le pide al Repository que agregue; limpia el campo (I).
3. Repository agrega la tarea (G) y la guarda en el archivo (H).

Repository = **dueño del dato** · ViewModel = **intermediario** (lo pide y lo prepara) · View = **solo lo muestra**. La lista "verdadera" vive en un solo lugar: **fuente de verdad única**.

Desafío: las dos flechas (UDF – flujo unidireccional de datos)



> **Evento vs. estado**
>
> - **Evento** = algo que *pasó* (un click, una tecla). Es puntual.
> - **Estado** = cómo *está* la pantalla ahora. Se puede consultar en cualquier momento.
> - **UDF:** los eventos suben y el estado baja. La View nunca modifica el estado por su cuenta: solo avisa y redibuja.

Preguntas de la guía

1. ¿Qué trabajo tendría que seguir funcionando aunque mañana cambiemos Compose por otra interfaz?
Las **reglas** (E, F: no vacía, no repetida) y el **manejo de datos** (G, H: agregar y guardar). Es decir, todo lo que va en ViewModel y Repository.

2. ¿Qué datos existen sólo mientras se está escribiendo un formulario?
`texto` (el borrador) y `error` (el mensaje actual). Son estado de pantalla y no se guardan en el archivo.

> **Señal de alarma: `Context` en un composable**
>
> `PantallaTareas` recibe un `Context` solo para poder escribir el archivo (H). La guía advierte no aceptar soluciones que "llevan Context a todos los composables" o "abren archivos desde un botón". Es una pista de que el acceso a archivos está en el lugar equivocado: debería estar en el Repository.

Kotlin / Compose que apareció en el camino

Elemento	Qué es
`Unresolved reference: X`	Kotlin no sabe qué es X: falta el `import`, está mal escrito o la función/clase no existe.
`var x by remember { mutableStateOf(...) }`	Estado observable que sobrevive a la recomposición. El `by` necesita los imports `getValue` / `setValue`.
`LocalContext.current`	Forma de obtener el `Context` de Android dentro de un composable.
`modifier: Modifier = Modifier`	Convención: permite que quien llama le pase padding, tamaño, etc. (por ejemplo el `innerPadding` del Scaffold).
`@Preview` / `@Composable`	Anotaciones: "etiquetas" que siempre se aplican a una función. Solas, sin función debajo, no compilan.

Ejercicio 2 — Una pantalla no es su estado

Consigna: pantalla de notas con título, contenido y botón Guardar. Definir una `data class` de estado de pantalla con el contenido actual, el error de validación y si hay una operación en curso. Simular el guardado. Identificar eventos separados para editar el título, editar el texto y pulsar Guardar. Todavía **sin ViewModel**.
Archivo: `domain/model/Nota.kt + presentation/nota/` · Rama: `ej02-estado-pantalla`

En pocas palabras

Se arma una pantalla de notas cuya apariencia depende de **un solo objeto** (`NotaUiState`) en vez de varias variables sueltas. Todavía no hay ViewModel: el objetivo es aprender a pensar la pantalla como "una foto" de su estado.

Importante a considerar

- **Inmutable no significa que no cambie la pantalla.** El estado no se modifica: se *reemplaza* por uno nuevo con `copy(...)`, y eso es lo que hace redibujar.
- **`copy` copia todo y cambia solo lo indicado:** `estado.copy(titulo = "x")` deja igual el contenido, el error, etc.
- **Lo escrito no es lo guardado.** `titulo` cambia en cada tecla; `guardada` solo al apretar Guardar. Por eso son dos campos.
- **`(String) -> Unit` es el tipo de una función** (recibe un texto, no devuelve nada). La pantalla recibe funciones para avisar lo que pasa.
- **Si la pantalla se puede previsualizar con cualquier estado armado a mano**, entonces depende solo de ese estado. Esa es la prueba del ejercicio.
- **`launch` + `delay`** espera sin congelar la app. Por ahora alcanza con esa idea; las corrutinas se ven más adelante.

Estado de pantalla (UiState)

Una sola "foto" de todo lo que la pantalla necesita para dibujarse. En el Ej 1 había 3 variables sueltas; ahora todo está junto en un objeto. **Si sabés el estado, sabés exactamente qué se ve.**

El estado

```
data class Nota(val titulo: String, val contenido: String)   // una nota YA guardada

data class NotaUiState(
    val titulo: String = "",          // lo escrito en el campo título
    val contenido: String = "",       // lo escrito en el campo contenido
    val error: String? = null,        // error de validación (null = sin error)
    val guardando: Boolean = false,   // operación en curso
    val guardada: Nota? = null        // última nota guardada (null = ninguna)
) {
    val hayCambiosSinGuardar: Boolean get() = ...   // propiedad CALCULADA
}
```

Elemento Kotlin	Por qué se usa
`data class`	Clase pensada para representar datos: genera `equals`, `toString` y `copy`.
Campos `val` + `copy(...)`	El estado es inmutable: para cambiarlo se crea una copia nueva con los campos modificados. Cada cambio es un estado nuevo y Compose detecta que tiene que redibujar.
`String?` / `Nota?`	Tipo anulable: el error o la nota guardada pueden no existir (`null`).
Valores por defecto (`= ""`)	`NotaUiState()` ya es el estado inicial: pantalla vacía.
Propiedad calculada (`get() = ...`)	Se deduce de otros campos en vez de guardarse: no hay información duplicada que pueda quedar desincronizada.

Desafío: tabla evento → cambio de estado (antes de codificar)

Evento	Cambio de estado esperado
Edita el **título**	`titulo = nuevo`, `error = null` (el error deja de tener sentido al escribir)
Edita el **contenido**	`contenido = nuevo`
Pulsa **Guardar** con título vacío	`error = "El título no puede estar vacío"`
Pulsa **Guardar** con título válido	`guardando = true`, `error = null`
Pulsa **Guardar** mientras ya guarda	Sin cambios: se ignora (y el botón está deshabilitado)
Termina el guardado simulado	`guardando = false`, `guardada = Nota(titulo, contenido)`

Cómo quedó organizado el código

Pieza	Rol
`Nota`	Modelo: una nota ya guardada.
`NotaUiState`	Estado de pantalla: **qué** se ve.
`alEditarTitulo`, `alEditarContenido`, `alPulsarGuardar`, `alTerminarGuardado`	Funciones Kotlin **puras**: reciben el estado actual y devuelven el nuevo. No conocen Compose. En el Ej 3 se mudan al ViewModel casi sin cambios.
`PantallaNotaConEstado`	Dueño del estado (provisorio, hasta el Ej 3) y del guardado simulado con `delay`.
`PantallaNota`	View **sin estado**: dibuja el `NotaUiState` que recibe y avisa los eventos por callbacks.
4 `@Preview`	Vacía · Error · Guardando · Guardada con cambios sin guardar.

State hoisting (elevar el estado)

```
@Composable
fun PantallaNota(
    estado: NotaUiState,                  // el estado BAJA
    onTituloChange: (String) -> Unit,     // los eventos SUBEN
    onContenidoChange: (String) -> Unit,
    onGuardar: () -> Unit,
    modifier: Modifier = Modifier
)
```

La View no guarda ni decide nada: recibe un valor y funciones (callbacks) para avisar. **Ventaja concreta:** como no tiene estado propio, se puede previsualizar en *cualquier* situación pasándole un `NotaUiState` armado a mano. Es el mismo UDF del Ej 1, ahora en código.

Lambdas y tipos función

`(String) -> Unit` = "una función que recibe un String y no devuelve nada". Se pasa como parámetro: `onTituloChange = { estado = alEditarTitulo(estado, it) }`. Dentro de la lambda, `it` es el único parámetro (el texto nuevo).

Guardado simulado: corrutina

`rememberCoroutineScope()` + `scope.launch { delay(1500); ... }` espera 1,5 s *sin congelar la pantalla*. Mientras tanto `guardando = true` → el botón muestra "Guardando..." y queda deshabilitado (`enabled = !estado.guardando`), lo que evita la doble pulsación. En el Ej 3 esto pasa a `viewModelScope`.

Preguntas de la guía

1. ¿El texto ingresado es lo mismo que la nota ya guardada?
No. `titulo` / `contenido` son lo que el usuario **está escribiendo** (cambia en cada tecla); `guardada` es la **última versión confirmada**. Por eso son campos distintos, y comparar ambos es lo que permite mostrar "Hay cambios sin guardar".

2. ¿Conviene mostrar el mensaje de error como parte del estado?
Sí. Si el error está en el estado, la pantalla se reconstruye completa a partir de `NotaUiState` (UI = f(estado)): sobrevive a la recomposición, se puede previsualizar y, con ViewModel (Ej 3), también a la rotación. Un mensaje "suelto" disparado desde el botón (un Toast) no se puede consultar ni recuperar.

Criterio de finalización

✅ Eventos reconocibles: `onTituloChange`, `onContenidoChange`, `onGuardar`.
✅ La pantalla se representa a partir de un único estado (`NotaUiState`): lo prueban las 4 previews, que dibujan cualquier situación sin ejecutar la app.

Explicación paso a paso

Recorrido del código del Ej 2, capa por capa: `domain/model/Nota.kt` (dato), `presentation/nota/NotaUiState.kt` (estado), `presentation/nota/NotaEventos.kt` (eventos) y `presentation/nota/PantallaNota.kt` (View).

Paso 0 · ¿Qué problema resuelve?

En el Ej 1 la pantalla tenía variables sueltas (`texto`, `tareas`, `error`): para saber qué se estaba viendo había que mirar tres cosas por separado, y cualquier parte del código podía cambiar cualquiera de ellas. La idea del Ej 2 es juntar todo en **un solo objeto**, el *estado de pantalla*. Si tenés ese objeto, sabés exactamente cómo se ve la pantalla: **UI = f(estado)**.

Paso 1 · `Nota`: el dato

```
data class Nota(
    val titulo: String,
    val contenido: String
)
```

Representa una **nota ya guardada**: el dato real, el que en ejercicios futuros va a ir al archivo. `data class` = clase cuyo propósito es guardar datos; Kotlin le genera `equals` (comparar), `toString` (imprimir) y `copy` (copiar cambiando algo). `val` = no se puede reasignar.

Paso 2 · `NotaUiState`: la "foto" de la pantalla

Campo	Pregunta que responde	Qué dibuja
`titulo`	¿Qué hay escrito en el campo título?	El texto dentro del campo
`contenido`	¿Qué hay escrito en el campo contenido?	El texto dentro del campo
`error`	¿Hay un error de validación?	El mensaje rojo debajo del título
`guardando`	¿Se está guardando ahora?	Botón "Guardando..." y deshabilitado
`guardada`	¿Cuál es la última nota guardada?	La tarjeta de abajo

- **`String?` y `Nota?`**: el `?` significa que puede ser `null` (puede no haber error, o ninguna nota guardada todavía).
- **Valores por defecto** (`= ""`, `= null`, `= false`): por eso `NotaUiState()` sin parámetros es el estado inicial, la pantalla vacía.
- **Todo es `val`**: el estado no se modifica, se **reemplaza por uno nuevo** con `copy`.

```
val antes = NotaUiState(titulo = "Hola")
val despues = antes.copy(error = "algo")   // mismo título, pero con error
// "antes" no cambió. "despues" es un objeto NUEVO.
```

¿Por qué así? Compose redibuja cuando el estado es **reemplazado por otro distinto**. Si se modificara un objeto por dentro, Compose podría no enterarse.

> **La propiedad calculada `hayCambiosSinGuardar`**
> No está entre los paréntesis: no se guarda, **se calcula** cada vez que alguien la lee, comparando `titulo` / `contenido` con `guardada`. Si fuera un campo más habría que acordarse de actualizarlo cada vez que cambia el título, el contenido o la nota guardada; un olvido y la pantalla muestra algo falso. Calculada, **no puede quedar desincronizada nunca**.

Paso 3 · Los eventos: "qué pasa cuando..."

Son el corazón del ejercicio. Todas tienen la misma forma: **reciben el estado actual → devuelven el estado nuevo**.

Función	Qué hace
`alEditarTitulo(estado, nuevo) = estado.copy(titulo = nuevo, error = null)`	Actualiza el título y **borra el error**: si decía "El título no puede estar vacío" y el usuario empieza a escribir, ese mensaje ya no tiene sentido.
`alEditarContenido(estado, nuevo) = estado.copy(contenido = nuevo)`	Solo cambia el contenido.
`alPulsarGuardar(estado)`	`when` evalúa en orden y usa la primera condición que se cumpla: 1) ¿ya se está guardando? → no hace nada; 2) ¿título vacío o solo espacios? → pone el error; 3) si no → `guardando = true`.
`alTerminarGuardado(estado)`	Deja de estar "guardando" y lo escrito pasa a ser la `guardada`.

> **Lo clave**
> Ninguna tiene `@Composable`, ni `Text`, ni nada de pantalla: **son Kotlin puro**. Son las reglas que en el Ej 1 dijimos que van al **ViewModel**. En el Ej 3 se mudan ahí casi sin cambios.

Paso 4 · `PantallaNota`: la View

```
@Composable
fun PantallaNota(
    estado: NotaUiState,                  // lo que tiene que dibujar  → el estado BAJA
    onTituloChange: (String) -> Unit,     // funciones para AVISAR     → los eventos SUBEN
    onContenidoChange: (String) -> Unit,
    onGuardar: () -> Unit,
    modifier: Modifier = Modifier
)
```

> **¿Qué es `(String) -> Unit`?**
> El **tipo** de una función: "recibe un String y no devuelve nada" (`Unit` ≈ `void` de Java). `() -> Unit` no recibe nada y no devuelve nada. A `PantallaNota` se le pasan **funciones como parámetros**, y ella las llama cuando el usuario hace algo.

Adentro, cada pieza lee del estado:

```
OutlinedTextField(
    value = estado.titulo,                              // muestra lo que dice el estado
    onValueChange = onTituloChange,                     // cuando el usuario escribe, AVISA
    isError = estado.error != null,                     // si hay error, borde rojo
    supportingText = { estado.error?.let { Text(it) } }  // y el mensaje
)
Button(
    onClick = onGuardar,                                // cuando aprieta, AVISA
    enabled = !estado.guardando                         // deshabilitado mientras guarda
) { Text(if (estado.guardando) "Guardando..." else "Guardar") }
```

Lo que NO hace: no valida, no guarda, no decide nada. Si el usuario escribe, no cambia el título: solo avisa "escribieron esto". Esto se llama **state hoisting** (elevar el estado): la pantalla no es dueña de su estado, lo recibe desde arriba.

Paso 5 · `PantallaNotaConEstado`: el dueño (provisorio)

```
@Composable
fun PantallaNotaConEstado(modifier: Modifier = Modifier) {
    var estado by remember { mutableStateOf(NotaUiState()) }
    val scope = rememberCoroutineScope()

    PantallaNota(
        estado = estado,
        onTituloChange = { estado = alEditarTitulo(estado, it) },
        onContenidoChange = { estado = alEditarContenido(estado, it) },
        onGuardar = { ... },
    )
}
```

- `mutableStateOf` : Compose **observa** esta variable; cuando cambia, redibuja.
- `remember` : hace que el valor **no se pierda** cuando Compose redibuja.
- `{ estado = alEditarTitulo(estado, it) }` es una **lambda** (función sin nombre); `it` es el parámetro que recibe (el texto nuevo). Se lee: "cuando cambie el título, reemplazá el estado por el resultado de `alEditarTitulo` ".
- En el Ej 3 este rol (dueño del estado) lo toma el **ViewModel**.

Paso 6 · El guardado simulado

```
onGuardar = {
    estado = alPulsarGuardar(estado)            // 1. aplicar la regla
    if (estado.guardando) {                     // 2. ¿pasó la validación?
        scope.launch {                          // 3. arrancar tarea en segundo plano
            delay(1500)                         // 4. esperar 1,5 s (simula el disco)
            estado = alTerminarGuardado(estado) // 5. listo
        }
    }
}
```

¿Por qué `scope.launch` ? Si se esperara 1,5 s "normalmente", **la pantalla se congelaría**. `launch` arranca una **corrutina**: una tarea que espera sin bloquear la pantalla. Por ahora alcanza con: *"launch + delay = esperar sin congelar la app"* (se profundiza en el Ej 9).

Paso 7 · Recorrido completo de un click

Título “Parcial” escrito, el usuario aprieta **Guardar**:

```
1. VIEW      Button → llama onGuardar()                              (EVENTO sube)
2. DUEÑO     estado = alPulsarGuardar(estado)
3. REGLA     título "Parcial" no está vacío → copy(guardando = true)
4. COMPOSE   detecta estado nuevo → redibuja PantallaNota
5. VIEW      lee estado.guardando = true → botón "Guardando..." deshabilitado
                                                                     (ESTADO baja)
   ... 1,5 segundos ...
6. DUEÑO     estado = alTerminarGuardado(estado)
7. REGLA     copy(guardando = false, guardada = Nota("Parcial", ""))
8. COMPOSE   redibuja
9. VIEW      botón vuelve a "Guardar" + aparece la tarjeta "Última nota guardada"
```

Nadie toca la pantalla directamente. Solo se cambia el estado, y la pantalla se redibuja sola a partir de él.

Paso 8 · Los Previews

```
@Preview(name = "2. Error de validación")
@Composable
fun PantallaNotaErrorPreview() {
    MvvmLabTheme {
        PantallaNota(
            NotaUiState(
                contenido = "Texto sin título",
                error = "El título no puede estar vacío"
            ),
            {}, {}, {}
        )
    }
}
```

Como `PantallaNota` solo dibuja lo que le pasan, se puede armar un estado a mano con cualquier situación y **verla sin correr la app**. Los `{}` son lambdas vacías: en el preview los botones no necesitan hacer nada. Es la **prueba del criterio de finalización**: si cualquier situación se dibuja pasando solo un `NotaUiState`, la pantalla se representa a partir de un único estado.

Resumen

1. **`NotaUiState`** = todo lo que la pantalla necesita, en un objeto. Inmutable: se cambia con `copy`.
2. **Funciones de evento** = reglas puras *estado → estado nuevo*. Sin Compose. En el Ej 3 van al ViewModel.
3. **`PantallaNota`** = View sin estado. Recibe `estado` (baja) y callbacks (suben). No decide nada.
4. **`PantallaNotaConEstado`** = dueño provisorio del estado. Conecta eventos con reglas.
5. **Nunca se toca la pantalla**: se cambia el estado y Compose redibuja.

Para comprobar que se entendió

Si en `alEditarTitulo` se sacara el `error = null` y quedara solo `estado.copy(titulo = nuevo)`, ¿qué pasaría en la pantalla al apretar Guardar con el título vacío y después empezar a escribir?

Ejercicio 3 — Primer ViewModel, sin magia

Consigna: mover la lógica del ejercicio anterior a una clase que extienda `ViewModel`. La pantalla recibe estado y funciones de evento, sin decidir cómo se guarda una nota. Obtener el ViewModel con el mecanismo apropiado del proyecto y comprobar qué sucede al rotar el emulador. **Desafío:** contador de intentos de guardado para observar la vida del ViewModel.
Rama: `ej03-viewmodel` · Archivos: `presentation/nota/NotaViewModel.kt` (ViewModel), `presentation/nota/NotaUiState.kt` (modelo y estado), `presentation/nota/PantallaNota.kt` (View)

> **En pocas palabras**
> El estado y las reglas se mudan del composable a un **ViewModel**. Así lo escrito **no se pierde al rotar el teléfono**, porque Android destruye la pantalla pero conserva el ViewModel.

> **Importante a considerar**
> - **Rotar = Android destruye y vuelve a crear la Activity.** Todo lo que estaba en `remember` se pierde; el ViewModel no.
> - **Nunca crear el ViewModel con `NotaViewModel()`.** Se obtiene con `by viewModels()`, que devuelve el mismo después de rotar.
> - **ViewModel ≠ guardar datos.** Si Android mata la app (o el usuario la cierra), el ViewModel muere con ella. Para guardar de verdad hacen falta archivos o base de datos.
> - `private set` permite leer la variable desde afuera pero cambiarla solo desde adentro: la pantalla no puede saltearse al ViewModel.
> - `viewModel::onGuardar` es "pasá la función onGuardar de este objeto". Equivale a `{ viewModel.onGuardar() }`.
> - **Logcat** es la consola de Android Studio. Filtrando por una etiqueta (`NotaViewModel`) se ve cuándo nace o muere el ViewModel.

Paso 0 · ¿Qué problema resuelve?

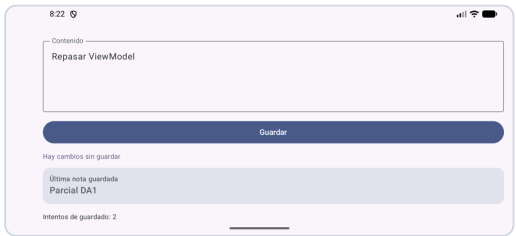
En el Ej 2 el dueño del estado era un composable con `remember`. `remember` vive mientras vive **la pantalla**. Pero cuando el teléfono **rota**, Android **destruye y vuelve a crear la Activity** (es un *cambio de configuración*; también pasa al cambiar idioma, modo oscuro o tamaño de ventana). Con la Activity se va todo lo que estaba en `remember`: **se borra lo escrito**.



Ej 2 (remember) antes de rotar



Ej 2 (remember) después de rotar: **se perdió todo**



Ej 3 (ViewModel) después de rotar: **se conserva todo** (intentos: 2)

Capturas reales del emulador (Pixel 8).

> **ViewModel**
> Clase que guarda el **estado de pantalla** y la **lógica** de una pantalla, y que **sobrevive a los cambios de configuración**. Android la mantiene viva mientras la pantalla "exista para el usuario", aunque la Activity se destruya y recree por dentro. No dibuja nada y no conoce la Activity.

Paso 1 · La clase `NotaViewModel`

```
class NotaViewModel : ViewModel() {          // ": ViewModel()" = hereda de ViewModel

    var estado by mutableStateOf(NotaUiState())
        private set                           // afuera se LEE, solo el VM lo CAMBIA

    var intentosGuardado by mutableStateOf(0) // desafío
        private set

    init { Log.d(TAG, "ViewModel CREADO") }   // se ejecuta UNA vez al crearse
    ...
}
```

- **`: ViewModel()`**: `NotaViewModel` *hereda* de la clase `ViewModel` de Android; eso es lo que le da la capacidad de sobrevivir a la rotación.
- **Mismo `NotaUiState` del Ej 2**: el estado no cambió, cambió *quién es su dueño*.
- **`private set`**: la pantalla puede leer `viewModel.estado` pero **no** puede hacer `viewModel.estado = ...`. Solo el ViewModel modifica su estado → UDF garantizado por el compilador. (En el Ej 4 esto se hace con `StateFlow`.)
- **`init { }`**: bloque que corre al construir el objeto. El `Log.d` permite ver en **Logcat** (filtro `NotaViewModel`) cuántas veces se crea.

Paso 2 · Los eventos ahora son métodos del ViewModel

```
fun onTituloChange(nuevo: String) {
    estado = estado.copy(titulo = nuevo, error = null)
}

fun onContenidoChange(nuevo: String) {
    estado = estado.copy(contenido = nuevo)
}

fun onGuardar() {
    if (estado.guardando) return                  // ya guardando: ignorar
    intentosGuardado++                            // desafío: cuenta TODOS los intentos
    if (estado.titulo.isBlank()) {
        estado = estado.copy(error = "El título no puede estar vacío")
        return
    }
    estado = estado.copy(guardando = true, error = null)
    viewModelScope.launch {
        delay(1500)                               // guardado SIMULADO
        estado = estado.copy(guardando = false,
                             guardada = Nota(estado.titulo, estado.contenido))
    }
}
```

Son **las mismas reglas** que en el Ej 2 eran funciones sueltas (`alEditarTitulo`, `alPulsarGuardar`...). Se "mudaron" al ViewModel casi sin cambios. Por eso valió la pena separarlas en el Ej 2.

> **`viewModelScope` en vez de `rememberCoroutineScope`**
> Las corrutinas lanzadas en `viewModelScope` viven lo que vive el ViewModel: si se rota *a mitad del guardado*, el guardado sigue y termina bien. Con `rememberCoroutineScope` (Ej 2) se cancelaba junto con la pantalla. Si el ViewModel se destruye de verdad, sus corrutinas se cancelan solas.

Paso 3 · `onCleared()`: el final de la vida

```
override fun onCleared() {
    Log.d(TAG, "ViewModel DESTRUIDO")
}
```

Android lo llama cuando el ViewModel se destruye **definitivamente** (por ejemplo, salir de la pantalla con "atrás"). **No** se llama al rotar.

Paso 4 · Obtener el ViewModel: `by viewModels()`

```
class MainActivity : ComponentActivity() {
    private val notaViewModel: NotaViewModel by viewModels()
    ...
        PantallaNotaConViewModel(viewModel = notaViewModel, ...)
}
```

- `by viewModels()` **no crea uno nuevo cada vez**: le pide a Android "el ViewModel de esta pantalla". La primera vez lo crea; si la Activity se recrea (rotación), devuelve **el mismo**.
- Es el "mecanismo apropiado del proyecto": viene en `activity-ktx`, que ya estaba como dependencia de `activity-compose`. **No hubo que agregar bibliotecas.**

> **Error típico: `val vm = NotaViewModel()`**
> Crearlo "a mano" lo convierte en un objeto común: se crea uno nuevo en cada recreación (o peor, en cada recomposición si se hace dentro de un composable) y el estado se pierde igual que con `remember`. La guía lo marca como propuesta a rechazar.

Paso 5 · La pantalla: `PantallaNota` se reutiliza SIN CAMBIOS

```
@Composable
fun PantallaNotaConViewModel(viewModel: NotaViewModel, modifier: Modifier = Modifier) {
    Column(modifier.verticalScroll(rememberScrollState())) {
        PantallaNota(
            estado = viewModel.estado,                    // el estado BAJA
            onTituloChange = viewModel::onTituloChange,   // los eventos SUBEN
            onContenidoChange = viewModel::onContenidoChange,
            onGuardar = viewModel::onGuardar
        )
        Text("Intentos de guardado: ${viewModel.intentosGuardado}")
    }
}
```

- **`PantallaNota` es exactamente la del Ej 2.** Como era una View sin estado, no le importa quién es el dueño (antes un composable, ahora un ViewModel). Este es el beneficio concreto del *state hoisting*.

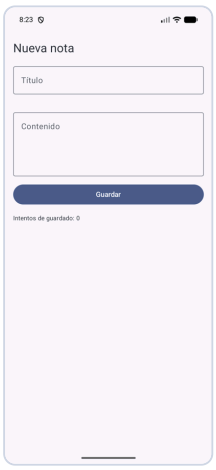
- `viewModel::onGuardar` es una *referencia a función*: “pasá el método `onGuardar` de este objeto”. Equivale a la lambda `{ viewModel.onGuardar() }`, pero más corta.
- `verticalScroll`: al rotar, en horizontal la pantalla no entra entera; sin scroll, la tarjeta y el contador quedaban fuera de vista.

Paso 6 · Quién hace qué (criterio de finalización)

Pieza	Hace	NO hace
MainActivity	Obtiene el ViewModel y lo conecta con la pantalla.	✅ No valida notas.
PantallaNota	Dibuja `estado`, avisa eventos.	No decide, no guarda.
NotaViewModel	Es dueño del estado, aplica reglas, coordina el guardado.	✅ No llama composables; no conoce la Activity.

Experimentos realizados

Prueba	Resultado observado
Rotar con **Ej 2** (`remember`)	Título y contenido **se borran**.
Rotar con **Ej 3** (ViewModel)	Se conservan título, contenido, nota guardada e **intentos: 2**. Logcat: `ViewModel CREADO` aparece **una sola vez** (rotando ida y vuelta).
App a segundo plano + `adb shell am kill` (Android mata el proceso) y volver	Pantalla **vacía**, intentos: 0. Logcat: un segundo `ViewModel CREADO` con **otro** identificador.



Después de matar el proceso:
ViewModel nuevo, estado perdido

¿Qué sobrevive a qué? (pregunta transversal de la guía)

	Recomposición	Rotación	Muerte del proceso
`remember`	✅	❌	❌
`rememberSaveable`	✅	✅	✅* (datos chicos)
ViewModel	✅	✅	❌
Repository + archivo / Room	✅	✅	✅

* Solo si el proceso lo mató *Android* (falta de memoria); si el usuario cierra la app a propósito, también se pierde.

Preguntas de la guía

1. ¿Qué problema resuelve el ViewModel?
Dos cosas: (a) **sobrevive a los cambios de configuración** (rotación, idioma, modo oscuro): el estado de pantalla y las operaciones en curso no se pierden cuando Android recrea la Activity; (b) **saca la lógica de la UI**: las reglas quedan en una clase Kotlin que no dibuja ni conoce la Activity, y la pantalla solo muestra estado y avisa eventos.

2. ¿Qué sucederá si el proceso se elimina completamente?
El ViewModel vive **en memoria**, dentro del proceso: si Android lo mata (por falta de memoria con la app en segundo plano) o el usuario la cierra, el ViewModel **muere con él** y al volver se crea uno nuevo, vacío. Se pierde todo (lo comprobamos). **Un ViewModel no es persistencia.** Para sobrevivir a eso: lo escrito a medias, con `SavedStateHandle` / `rememberSaveable`; los datos reales, con un Repository que guarde en archivo o Room (Ej 6 en adelante).

Resumen

1. El estado y las reglas se mudan del composable al **ViewModel**; `NotaUiState` y `PantallaNota` no cambian.
2. El ViewModel **sobrevive a la rotación** pero **no a la muerte del proceso**.
3. Se obtiene con `by viewModels()`, nunca con `NotaViewModel()` a mano.
4. `private set`: la pantalla lee el estado pero no puede cambiarlo.
5. `viewModelScope`: corrutinas que viven lo que vive el ViewModel.

Ejercicio 4 — StateFlow: la pantalla observa

Consigna: exponer desde el ViewModel un `StateFlow` de sólo lectura y mantener mutable su implementación. En Compose recolectarlo con la API consciente del ciclo de vida (`collectAsStateWithLifecycle`). Hacer que editar un campo cambie el estado visible. **Desafío:** registrar cambios puntuales y comprobar que editar un campo no borra el otro.
Rama: `ej04-stateflow` · Archivo principal: `presentation/nota/NotaViewModel.kt`

> **En pocas palabras**
> El ViewModel pasa a exponer su estado con **StateFlow**, una herramienta de Kotlin (no de Compose) para valores que cambian. La pantalla se "suscribe" y se redibuja sola cuando llega un estado nuevo.

> **Importante a considerar**
> - **El guion bajo (`_uiState`) es una convención:** la versión privada que se puede modificar. La pública (`uiState`) es de solo lectura.
> - **`.value`** da el valor actual del StateFlow; **`update { }`** lo reemplaza a partir del actual, sin pisar cambios simultáneos.
> - **`collectAsStateWithLifecycle()`** convierte el flujo en algo que Compose observa y deja de escuchar cuando la app está en segundo plano.
> - **No guardar una copia del estado en la pantalla** (por ejemplo, otro `remember` con el título): habría dos versiones y podrían diferir.
> - **Se agregó una dependencia** (`lifecycle-runtime-compose`) porque esa función no venía en el proyecto. Siempre hay que justificar por qué.

Paso 0 · ¿Qué cambia respecto del Ej 3?

En el Ej 3 el estado era un `mutableStateOf` (una API **de Compose**) dentro del ViewModel. Funciona, pero ata el ViewModel a Compose. `StateFlow` es una API **de Kotlin (corrutinas)**: el ViewModel queda independiente de la interfaz, y es lo que usan Room, los Repository y casi toda la documentación oficial.

> **StateFlow**
> Un "contenedor observable" que **siempre tiene un valor actual** (`.value`) y **avisa a quien lo observe** cada vez que cambia. Es el patrón *Observer* de la teoría: el ViewModel produce, la pantalla se suscribe y reacciona. `MutableStateFlow` se puede escribir; `StateFlow` solo se puede leer.

Paso 1 · Backing property: mutable adentro, solo lectura afuera

```
class NotaViewModel : ViewModel() {
    private val _uiState = MutableStateFlow(NotaUiState())     // PRIVADO: solo el VM escribe
    val uiState: StateFlow<NotaUiState> = _uiState.asStateFlow() // PÚBLICO: solo lectura
}
```

- Por convención, el guion bajo (`_uiState`) marca la versión privada y mutable.
- `asStateFlow()` devuelve una vista de **solo lectura** del mismo flujo: aunque alguien intente castearla, no puede escribir.
- Es la versión "flujo" del `private set` del Ej 3.

Paso 2 · Los eventos usan `update { }`

```
fun onTituloChange(nuevo: String) {
    _uiState.update { it.copy(titulo = nuevo, error = null) }
}
fun onContenidoChange(nuevo: String) {
    _uiState.update { it.copy(contenido = nuevo) }
}
```

`update { actual -> nuevo }` toma el valor **actual** y lo reemplaza por el que devuelve la lambda (`it` = estado actual). Cada evento hace `copy` **solo del campo que le corresponde** → editar el título nunca borra el contenido (desafío). Además `update` es **atómico**: si dos cambios llegan casi juntos, ninguno pisa al otro.

Paso 3 · La pantalla observa con `collectAsStateWithLifecycle`

```
@Composable
fun PantallaNotaConViewModel(viewModel: NotaViewModel, modifier: Modifier = Modifier) {
    val estado by viewModel.uiState.collectAsStateWithLifecycle()
    PantallaNota(
        estado = estado,
        onTituloChange = viewModel::onTituloChange,
        onContenidoChange = viewModel::onContenidoChange,
        onGuardar = viewModel::onGuardar
    )
}
```

- **collect** = suscribirse al flujo; **AsState** = convertir cada valor en un `State` de Compose → cuando llega uno nuevo, se recompone.
- **WithLifecycle** = deja de recolectar cuando la app pasa a segundo plano (no gasta trabajo en una pantalla que nadie ve) y retoma al volver. La versión sin ciclo de vida, `collectAsState()`, seguiría recolectando en segundo plano.
- Compose **no guarda una segunda copia** de la nota (ningún `remember` con el título): lee siempre la del ViewModel (criterio de finalización).
- `PantallaNota` y `NotaUiState` **no cambiaron**.

> **Dependencia agregada (justificada)**
> `collectAsStateWithLifecycle` vive en `androidx.lifecycle:lifecycle-runtime-compose`, que el proyecto no tenía. Se agregó en `libs.versions.toml` con versión `2.10.0`, la **misma** de lifecycle que ya usaba el proyecto (no se actualizó nada más). La guía la nombra explícitamente como la API esperada.

Experimento: registrar cambios puntuales (Logcat, filtro `NotaViewModel`)

```
titulo -> titulo='Parcial D'   contenido='Repasar StateFlow' error=null guardando=false
titulo -> titulo='Parcial DA'  contenido='Repasar StateFlow' error=null guardando=false
titulo -> titulo='Parcial DA1' contenido='Repasar StateFlow' error=null guardando=false
```

Se escribió el contenido y *después* se siguió editando el título: el contenido se mantiene en cada cambio. ✅

Preguntas de la guía

1. ¿Por qué la pantalla no debería poder modificar directamente `_uiState`?
Porque rompería el flujo unidireccional: el ViewModel dejaría de ser el único dueño del estado y cualquier pantalla podría saltearse las reglas (por ejemplo, poner `guardando = false` a mano o guardar un título vacío). Si solo el ViewModel escribe, cada cambio pasa por un evento con nombre (`onGuardar`...) y es fácil saber *quién* cambió *qué*. El compilador lo garantiza: `_uiState` es privado.

2. ¿Qué parte se recompone?
Cuando llega un estado nuevo se vuelve a ejecutar `PantallaNotaConViewModel` (es quien lee `estado`) y, dentro de `PantallaNota`, Compose redibuja solo lo que depende de valores que cambiaron: al escribir en el título se actualiza ese `OutlinedTextField` y el aviso de "cambios sin guardar"; partes cuyos parámetros no cambiaron (el texto fijo "Nueva nota", la tarjeta de la nota guardada) se *saltean*.

> **Resumen**
>
> 1. `_uiState: MutableStateFlow` privado + `uiState: StateFlow` público de solo lectura.
> 2. Eventos con `_uiState.update { it.copy(...) }`: atómico y sin pisar otros campos.
> 3. La pantalla recolecta con `collectAsStateWithLifecycle()` y no guarda copias propias.
> 4. StateFlow es Kotlin, no Compose: el ViewModel no depende de la UI.

Ejercicio 5 — Validar sin esconder la regla en un botón

Consigna: agregar reglas: título no vacío, contenido de al menos diez caracteres y prohibición de guardar dos veces mientras se procesa una solicitud. Ubicar las reglas fuera de la vista. Mostrar errores cerca de los campos, sin diálogos para cada tecla. **Desafío:** probar espacios en blanco, una nota válida y dos pulsaciones seguidas.
Rama: `ej05-validacion` · Archivos nuevos: `domain/validacion/ValidadorNota.kt`, `domain/validacion/ValidadorNotaTest.kt`

> **En pocas palabras**
> Se agregan reglas (título obligatorio, contenido mínimo, no guardar dos veces) y se ubican en una clase de **Kotlin puro**, fuera de la pantalla. Así se pueden probar con tests sin abrir el emulador.

> **Importante a considerar**
> - **`isBlank()` vs `isEmpty()`:** `" "` no está vacío pero sí está "en blanco". Para validar texto conviene `isBlank()`.
> - **`object`** crea una clase con una sola instancia; sirve para funciones sin estado como `ValidadorNota.validar(...)`.
> - **Deshabilitar el botón no alcanza como regla:** es una ayuda visual. La regla ("no guardar dos veces") tiene que estar en el ViewModel.
> - **Un error por campo** permite mostrar cada mensaje al lado de su campo y borrarlo cuando el usuario lo corrige.
> - **Test unitario:** función con `@Test` que prepara, actúa y verifica. Corre en la computadora en segundos (click derecho → Run).

Paso 0 · El problema: la regla "escondida" en el botón

Si la validación se escribe dentro del `onClick` del botón (como en la pantalla del Ej 1), la regla queda mezclada con la interfaz: no se puede probar sin abrir la pantalla, no se puede reutilizar y, si mañana se guarda desde otro lugar (otro botón, un atajo), hay que copiarla. La regla es **del negocio**, no del botón.

Paso 1 · `ValidadorNota`: las reglas en Kotlin puro

```
data class ErroresNota(val titulo: String? = null, val contenido: String? = null) {
    val hayErrores: Boolean get() = titulo != null || contenido != null
}

object ValidadorNota {
    const val MIN_CARACTERES_CONTENIDO = 10

    fun validar(titulo: String, contenido: String): ErroresNota = ErroresNota(
        titulo = if (titulo.isBlank()) "El título no puede estar vacío" else null,
        contenido = if (contenido.trim().length < MIN_CARACTERES_CONTENIDO)
            "El contenido debe tener al menos $MIN_CARACTERES_CONTENIDO caracteres" else null
    )
}
```

- **`object`**: un *singleton*, una clase con una única instancia. Sirve para funciones sin estado: se llama directo, `ValidadorNota.validar(...)`.

- **`isBlank()`** vs `isEmpty()`: `" ".isEmpty()` es `false`, pero `" ".isBlank()` es `true`. Con `isBlank` un título de solo espacios es inválido.
- **`trim()`** saca los espacios de los extremos antes de contar: `" corto "` tiene 11 caracteres pero solo 5 reales.
- **`ErroresNota`** tiene un error *por campo*: así cada mensaje se puede mostrar al lado de su campo.
- **`const val`**: constante conocida en compilación; el número 10 está en un solo lugar.

Paso 2 · El ViewModel aplica las reglas; el botón solo avisa

```
fun onGuardar() {
    val actual = _uiState.value
    if (actual.guardando) return                                    // regla 3: no guardar dos
veces

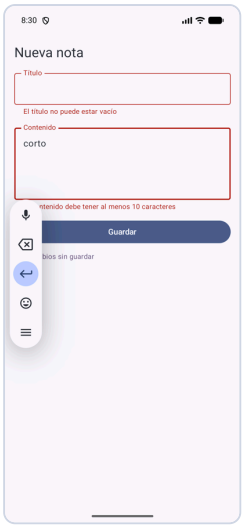
    val errores = ValidadorNota.validar(actual.titulo, actual.contenido) // reglas 1 y 2
    if (errores.hayErrores) {
        _uiState.update { it.copy(errorTitulo = errores.titulo, errorContenido =
errores.contenido) }
        return
    }
    _uiState.update { it.copy(guardando = true) }
    viewModelScope.launch { delay(1500); _uiState.update { it.copy(guardando = false, guardada =
...) } }
}
```

> **Regla 3: dos capas de protección**
> `enabled = !estado.guardando` en el botón es la ayuda **visual** (el usuario ve que no puede tocar). `if (actual.guardando) return` en el ViewModel es la **regla**: aunque llegaran dos clics antes de que el botón se deshabilite, el segundo se ignora. La regla no puede depender de que la vista se dibuje a tiempo.

Paso 3 · Estado: un error por campo

`NotaUiState` reemplaza `error` por `errorTitulo` y `errorContenido`. La vista solo decide *cómo* mostrarlos:

```
OutlinedTextField(
    value = estado.contenido, onValueChange = onContenidoChange,
    isError = estado.errorContenido != null,                    // borde rojo
    supportingText = { estado.errorContenido?.let { Text(it) } } // mensaje debajo del campo
)
```



Título con solo espacios y contenido “corto”:
cada error debajo de su campo

Paso 4 · ¿Cuándo se limpia un error?

```
fun onTituloChange(nuevo: String) {
    _uiState.update { it.copy(titulo = nuevo, errorTitulo = null) }   // solo el error del
título
}
```

Cuando el usuario **edita ese campo**: indica que lo está corrigiendo y no lo molesta en cada tecla. Se limpia **solo el error del campo editado**; el del otro campo sigue visible hasta que lo corrija. La validación completa se vuelve a hacer al pulsar Guardar.

Paso 5 · Tests del validador (desafío)

`app/src/test/.../ValidadorNotaTest.kt` — tests JUnit que corren en la computadora, **sin emulador** (click derecho → Run):

Test	Comprueba	Resultado
`tituloSoloEspacios_esInvalido`	`" "` no es un título válido	✅
`contenidoConEspaciosAlrededor_noCuentaLosEspacios`	`" corto "` no alcanza los 10	✅
`contenidoDeExactamente10Caracteres_esValido`	Borde: 10 caracteres es válido	✅
`notaValida_noTieneErrores`	Una nota correcta pasa	✅
`ambosCamposMal_informaAmbosErrores`	Se informan los dos errores juntos	✅

Dos pulsaciones seguidas (en el emulador, dos toques casi simultáneos): Logcat muestra un único `guardando '...'` → se guardó una sola vez. ✅

Preguntas de la guía

1. ¿Cuál es una regla de negocio y cuál es una decisión visual?
Regla de negocio: “el contenido debe tener al menos 10 caracteres”, “no se guarda dos veces a la vez” — valen igual en Android, web o una API. *Decisión visual*: mostrar el error en rojo debajo del campo, deshabilitar el botón, el texto “Guardando...”. Si cambio la interfaz, las decisiones visuales cambian; las reglas no.

2. ¿En qué momento conviene limpiar un mensaje de error?
Cuando el usuario modifica el campo que lo causó (y solo ese error). Limpiarlo antes (o validar en cada tecla mostrando errores desde la primera letra) molesta; no limpiarlo nunca deja un mensaje desactualizado mientras el usuario ya lo está corrigiendo.

> **Resumen**
>
> 1. Las reglas viven en `ValidadorNota` (Kotlin puro) → se prueban sin emulador.
> 2. El botón solo comunica la intención; el ViewModel aplica las reglas.
> 3. “No guardar dos veces” es una regla del ViewModel; el botón deshabilitado es solo ayuda visual.
> 4. Un error por campo, mostrado debajo de su campo, y limpiado al editar ese campo.

Ejercicio 6 — Repository: cambiar datos sin cambiar la pantalla

Consigna: definir un contrato `NotaRepository` con operaciones para obtener, guardar y eliminar notas. Implementar primero una versión en memoria. El ViewModel sólo habla con el contrato. Decidir si el resultado se devuelve como valor puntual o como `Flow` y justificarlo. **Desafío:** reemplazar la implementación por otra lista precargada sin modificar el composable.
Rama: `ej06-repository` · Archivo nuevo: `domain/repository/NotaRepository.kt`

> **En pocas palabras**
> Aparece el **Repository**: una pieza que se encarga de los datos (guardar, listar, eliminar) detrás de una **interface**. El ViewModel solo conoce la interface, así que se puede cambiar dónde se guardan los datos sin tocar la pantalla.

> **Importante a considerar**
> - **Interface = promesa.** Dice "existe `guardar(nota)`", no cómo. Distintas clases la cumplen a su manera (memoria, archivo, Room).
> - `override` marca que esa función es la que pide la interface.
> - **Cada nota necesita un `id`** para poder eliminarla o editarla. Se genera con `UUID.randomUUID()`, un texto único al azar.
> - `suspend` en el Repository avisa que la operación puede tardar; por eso se llama dentro de `viewModelScope.launch`.
> - `LazyColumn` es una lista con scroll que solo dibuja las filas visibles. `key = { it.id }` ayuda a no mezclar filas al borrar.
> - **"En memoria" significa que se pierde al cerrar la app.** Es un primer paso para probar la pantalla; después vienen archivos y Room.

Paso 0 · ¿Para qué un Repository?

Hasta el Ej 5 el ViewModel "simulaba" guardar. Pero el ViewModel no debería saber *dónde* se guardan los datos (memoria, archivo, base de datos, internet): su trabajo es coordinar la pantalla. El **Repository** es la pieza de la capa de datos que esconde ese "dónde" detrás de un contrato. Es la **(G)** y la **(H)** de la tabla del Ej 1.

> **Contrato = `interface`**
> Una `interface` dice **qué** operaciones existen, sin decir **cómo** se hacen. Varias clases pueden *implementarla* (`: NotaRepository`), cada una a su manera. Quien usa la interfaz no sabe ni le importa cuál le tocó.

Paso 1 · El contrato

```
interface NotaRepository {
    suspend fun obtenerTodas(): List<Nota>
    suspend fun guardar(nota: Nota)      // si ya existe ese id, la reemplaza
    suspend fun eliminar(id: String)
}
```

- **`suspend`**: la operación puede tardar (disco, red) y se ejecuta dentro de una corrutina sin bloquear la pantalla. Solo se puede llamar desde otra función `suspend` o desde `launch`.
- **`Nota` ahora tiene `id`**: con varias notas, el Repository necesita saber cuál eliminar o reemplazar. El ViewModel lo genera con `UUID.randomUUID().toString()` (un identificador único al azar).

Paso 2 · Dos implementaciones del mismo contrato

```
class NotaRepositoryEnMemoria : NotaRepository {           // empieza vacía
    private val notas = mutableListOf<Nota>()
    override suspend fun obtenerTodas(): List<Nota> { delay(300); return notas.toList() }
    override suspend fun guardar(nota: Nota) {
        delay(300)
        val i = notas.indexOfFirst { it.id == nota.id }
        if (i >= 0) notas[i] = nota else notas.add(nota)
    }
    override suspend fun eliminar(id: String) { delay(300); notas.removeAll { it.id == id } }
}

class NotaRepositoryDeEjemplo : NotaRepository {           // desafío: precargada
    private val notas = mutableListOf(
        Nota("ej-1", "Parcial DA1", "Repasar MVVM, StateFlow y Repository"), ...)
    ...
}
```

- **`override`**: “esta función cumple la que pide la interfaz”.
- **`notas.toList()`** devuelve una *copia*: quien la recibe no puede modificar la lista interna del Repository.
- `delay(300)` simula una fuente que tarda, para ver que la pantalla sigue respondiendo.

Paso 3 · El ViewModel recibe el contrato

```
class NotaViewModel(
    private val repositorio: NotaRepository = NotaRepositoryEnMemoria()   // ← único lugar donde
se elige
) : ViewModel() {

    init { viewModelScope.launch { recargarNotas() } }

    private suspend fun recargarNotas() {
        val notas = repositorio.obtenerTodas()
        _uiState.update { it.copy(notas = notas) }
    }

    fun onGuardar() {
        ... // validación igual que en el Ej 5
        viewModelScope.launch {
            repositorio.guardar(Nota(UUID.randomUUID().toString(), titulo, contenido))
            recargarNotas()
            _uiState.update { it.copy(guardando = false, titulo = "", contenido = "") }
        }
    }

    fun onEliminar(id: String) {
        viewModelScope.launch { repositorio.eliminar(id); recargarNotas() }
    }
}
```

- El tipo del parámetro es `NotaRepository` (la interfaz), **no** `NotaRepositoryEnMemoria`: el ViewModel solo habla con el contrato.
- El valor por defecto `= NotaRepositoryEnMemoria()` es provisorio. Como todos los parámetros tienen valor por defecto, Kotlin genera un constructor vacío y `by viewModels()` lo puede crear. En el **Ej 11** esto se reemplaza por una fábrica explícita.

Paso 4 · La pantalla: lista con `LazyColumn`

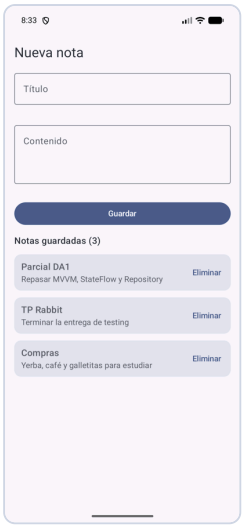
```
LazyColumn(...) {
    item { /* formulario: título, contenido, Guardar, "Notas guardadas (n)" */ }
    items(estado.notas, key = { it.id }) { nota ->
        FilaNota(nota = nota, onEliminar = { onEliminar(nota.id) })
    }
}
```

- `LazyColumn` solo dibuja las filas visibles (como un `RecyclerView`). `item { }` agrega un elemento suelto (el formulario); `items(lista) { }`, uno por cada nota.
- `key = { it.id }`: Compose identifica cada fila por su id, no por su posición; al borrar una no se "corren" las demás.
- `FilaNota` es un composable reutilizable para una fila. La pantalla recibe `List<Nota>` ya lista: **no conoce al Repository**.

Decisión: valor puntual (`suspend`) y no `Flow`

	Valor puntual (`suspend fun obtenerTodas(): List`)	**Flow (`fun observar(): Flow<List>`)**
Qué devuelve	Una "foto" de los datos en ese momento	Un flujo que emite cada vez que los datos cambian
Después de escribir	El ViewModel vuelve a pedir la foto (`recargarNotas()`)	Se actualiza solo
Conviene cuando	Una sola pantalla lee y escribe	Varias pantallas miran los mismos datos, o la fuente ya emite (Room)

Elegimos valor puntual: es más simple y explícito, y hoy hay una sola pantalla. En el **Ej 12** (dos pantallas) pasamos a `Flow`.



Desafío: con `NotaRepositoryDeEjemplo()`
aparecen 3 notas sin tocar la pantalla

Pruebas en el emulador

Prueba	Resultado
Agregar “Parcial DA1” y “Compras”	“Notas guardadas (2)”, ambas en la lista ✅
Eliminar “Parcial DA1”	“Notas guardadas (1)”, queda “Compras” ✅
Cambiar *una línea* del ViewModel a `NotaRepositoryDeEjemplo()`	Arranca con 3 notas precargadas; `presentation/nota/PantallaNota.kt` sin cambios ✅

Preguntas de la guía

1. ¿Qué conoce el Repository sobre Compose?
Nada. No importa ninguna clase de Compose ni de la UI: recibe y devuelve `Nota` (una `data class` de Kotlin). Podría usarse igual desde otra interfaz, un test o un servicio en segundo plano.

2. ¿Por qué esta interfaz ayuda a probar la app?
Porque el ViewModel depende del contrato, no de una implementación: en un test se le puede pasar un Repository *falso* con datos prefijados o que falle a propósito, sin archivos ni base de datos, y comprobar las reglas del ViewModel rápido y de forma repetible (eso es el **Ej 13**).

> **Resumen**
>
> 1. `interface NotaRepository` = contrato: qué operaciones hay, no cómo.
> 2. El ViewModel recibe `NotaRepository` por constructor y solo habla con el contrato.
> 3. Cambiar de implementación = cambiar una línea; la pantalla no se entera.
> 4. Valor puntual + recargar después de escribir: simple mientras haya una sola pantalla.

Ejercicio 7 — Leer un archivo de texto desde el Repository

Consigna: crear `ArchivoTextoRepository` para leer `borrador.txt` desde `filesDir`. Si no existe, devolver texto vacío o un resultado explícito (documentar la decisión). El ViewModel solicita la lectura y publica el resultado. No abrir el archivo desde un composable. Distinguir archivo inexistente de una falla real; usar `Dispatchers.IO`; no guardar referencias a la Activity. **Desafío:** inspeccionar el archivo con Device Explorer.
Rama: `ej07-leer-archivo` · Archivos: `domain/repository/BorradorRepository.kt` + `data/repository/ArchivoTextoRepository.kt`, `presentation/borrador/BorradorViewModel.kt`, `presentation/borrador/PantallaBorrador.kt`

En pocas palabras

Se lee un archivo de texto (`borrador.txt`) desde la carpeta privada de la app, siempre a través del Repository. Se distinguen tres casos: el archivo no existe, se leyó bien, o hubo un error real.

Importante a considerar

- **`filesDir`** es una carpeta que solo ve nuestra app. No hace falta pedir permisos y se borra al desinstalar.
- **"No existe" no es un error:** es lo normal la primera vez. Por eso se devuelve un resultado (`NoExiste`), y los errores reales se avisan con una excepción.
- **`sealed interface`** define un conjunto cerrado de casos posibles. El compilador obliga a contemplarlos todos en un `when`.
- **`Dispatchers.IO`** manda el trabajo de disco a otros hilos. Leer archivos en el hilo principal puede congelar la pantalla.
- **El Repository recibe la carpeta (`File`), no la Activity.** Guardar una Activity en un objeto que vive más que ella provoca fugas de memoria.
- **La fábrica (`Factory`)** existe porque ahora el ViewModel necesita parámetros y Android no sabe construirlo solo.

Cambio de pantalla

Los Ej 7, 8 y 9 de la guía trabajan sobre **un único borrador de texto**; en el Ej 10 se vuelve a varias notas. Por eso en esta rama la app es un editor de borrador (las notas, el validador y su Repository viven en las ramas anteriores).

Paso 0 · ¿Dónde se guarda? `filesDir`

Cada app tiene una carpeta **privada** en el almacenamiento interno: `/data/data/com.example.mvvmlab/files/` (se obtiene con `context.filesDir`). No hace falta pedir permisos, nadie más puede leerla y se borra al desinstalar la app.

Paso 1 · Decisión documentada: resultado explícito

```
sealed interface ResultadoLectura {
    data object NoExiste : ResultadoLectura
    data class Leido(val texto: String) : ResultadoLectura
}
```

Si el archivo no existe se devuelve `NoExiste` y **no** un texto vacío: “no existe” y “existe pero está vacío” son situaciones distintas, y con `""` no habría forma de diferenciarlas. Una **falla real** (permiso, disco, archivo dañado) no es un resultado: se informa con una **excepción** (`IOException`).

> **`sealed interface` (teoría, diapositiva 12)**
> Restringe los subtipos a un conjunto **conocido por el compilador**. Un `when` sobre un `ResultadoLectura` es *exhaustivo*: si mañana se agrega un caso, el compilador marca todos los `when` que no lo contemplan. `data object` = un caso sin datos; `data class` = un caso con datos.

Paso 2 · El Repository lee en `Dispatchers.IO`

```
interface BorradorRepository {
    suspend fun leer(): ResultadoLectura
}

class ArchivoTextoRepository(carpeta: File) : BorradorRepository {
    private val archivo = File(carpeta, "borrador.txt")

    override suspend fun leer(): ResultadoLectura =
        withContext(Dispatchers.IO) {
            if (!archivo.exists()) ResultadoLectura.NoExiste
            else ResultadoLectura.Leido(archivo.readText(Charsets.UTF_8))   // puede lanzar
IOException
        }
}
```

- **`withContext(Dispatchers.IO)`**: ejecuta el bloque en hilos pensados para disco y red. Leer un archivo en el hilo principal puede congelar la pantalla (y Android lo detecta como mala práctica).
- **Recibe un `File` (la carpeta), no un `Context` ni una Activity**: es lo único que necesita. Si guardara la Activity, el Repository la mantendría viva después de que Android la destruya (*fuga de memoria*).
- **`Charsets.UTF_8`**: se indica la codificación para que tildes y ñ se lean bien.

Paso 3 · El ViewModel solicita y publica

```
class BorradorViewModel(private val repositorio: BorradorRepository) : ViewModel() {
    init {
        viewModelScope.launch {
            try {
                when (val r = repositorio.leer()) {
                    is ResultadoLectura.Leido -> _uiState.update { it.copy(texto = r.texto,
mensaje = "Cargado desde borrador.txt") }
                    ResultadoLectura.NoExiste -> _uiState.update { it.copy(mensaje = "Todavía no
hay borrador guardado") }
                }
            } catch (e: IOException) {
                _uiState.update { it.copy(mensaje = "", error = "No se pudo leer el borrador") }
            }
        }
    }
}
```

`when (val r = ...)` guarda el resultado en `r` y elige la rama según su tipo; dentro de `is ResultadoLectura.Leido`, Kotlin ya sabe que `r` tiene `texto` (*smart cast*). La pantalla nunca toca el sistema de archivos: recibe `BorradorUiState`.

Paso 4 · La fábrica: cómo se construye un ViewModel con parámetros

```
companion object {
    val Factory: ViewModelProvider.Factory = viewModelFactory {
        initializer {
            val app = checkNotNull(this[APPLICATION_KEY])
            BorradorViewModel(ArchivoTextoRepository(app.filesDir))
        }
    }
}
// En MainActivity:
private val borradorViewModel: BorradorViewModel by viewModels { BorradorViewModel.Factory }
```

Ahora el ViewModel **recibe** un repositorio que necesita `filesDir`, y Android no sabe crearlo solo. La **fábrica** le explica cómo. Usa el `Application` (vive lo que vive la app, no una pantalla) solo para obtener `filesDir`. `viewModelFactory` ya venía en `lifecycle-viewmodel`: no se agregaron dependencias. En el **Ej 11** esto se ordena en un único lugar.

Pruebas en el emulador

Prueba	Resultado
Abrir sin `borrador.txt`	“Todavía no hay borrador guardado” (caso `NoExiste`) ✅
Crear el archivo (`adb shell run-as ... printf ... > files/borrador.txt`) con tres líneas, tildes y “ñandú”, y reabrir	El texto aparece completo al abrir la pantalla ✅
Falla real: reemplazar `borrador.txt` por una *carpeta*	“No se pudo leer el borrador”; Logcat: `FileNotFoundException ... EISDIR (Is a directory)`. La app no se cierra ✅



Borrador escrito previamente en el archivo,
cargado al abrir la pantalla

> **Desafío: Device Explorer**
> En Android Studio: *View* → *Tool Windows* → *Device Explorer* → `data/data/com.example.mvvmlab/files/borrador.txt`. Doble click lo abre; click derecho → *Upload* permite subir un archivo propio para probar. Equivalente por consola: `adb shell run-as com.example.mvvmlab cat files/borrador.txt`.

Preguntas de la guía

1. ¿Dónde está físicamente el archivo?
En el almacenamiento interno del dispositivo, en la carpeta privada de la app: `/data/data/com.example.mvvmlab/files/borrador.txt` (también visible como `/data/user/0/...`). Se borra al desinstalar la app.

2. ¿Otra app puede leerlo sin más?
No. Cada app corre con su propio usuario de Linux y la carpeta `/data/data/<paquete>` solo es accesible para ese usuario (*sandbox*). Otra app no puede abrirla aunque conozca la ruta; para compartir el contenido habría que hacerlo explícitamente (por ejemplo con un Intent, Ej 15).

> **Resumen**
> 1. `filesDir` = carpeta privada de la app, sin permisos.
> 2. Tres casos distintos: `NoExiste`, `Leido(texto)` y excepción (falla real).
> 3. El Repository lee en `Dispatchers.IO` y recibe la carpeta, no la Activity.
> 4. El ViewModel pide y publica; la UI no toca archivos. Una fábrica construye el ViewModel con su Repository.

Ejercicio 8 — Guardar texto, cerrar y volver

Consigna: agregar al Repository la escritura del borrador en UTF-8. Escribir varias líneas, guardar, cerrar completamente la app y volver a abrirla: debe recuperarse el contenido. Elegir y documentar si se sobrescribe todo el archivo o se agrega al final. **Desafío:** botón “Descartar cambios” que vuelva a leer la versión guardada.
Rama: `ej08-guardar-archivo`

> **En pocas palabras**
> Se agrega guardar el borrador en el archivo. Al cerrar la app y volver a abrirla, el texto sigue ahí. La pantalla muestra si hay cambios sin guardar y permite descartarlos.

> **Importante a considerar**
> - **Sobrescribir vs agregar al final:** un borrador es “el texto actual”, así que se reemplaza todo. Agregar al final sirve para un registro de eventos.
> - **UTF-8 al escribir y al leer.** Si no coincide la codificación, las tildes aparecen como caracteres raros (“CanciÃ³n”).
> - **“Guardado” se muestra recién cuando la escritura terminó bien.** Si falla, el usuario tiene que saberlo.
> - **Memoria vs disco:** lo que está en el editor vive en el ViewModel; lo guardado vive en el archivo. La diferencia entre ambos es “cambios sin guardar”.
> - **Cerrar del todo la app** (no solo minimizarla) es la prueba real de que algo persiste.

Paso 1 · El contrato crece: `guardar(texto)`

```
interface BorradorRepository {
    suspend fun leer(): ResultadoLectura
    suspend fun guardar(texto: String)
}

override suspend fun guardar(texto: String) {
    withContext(Dispatchers.IO) {                    // fuera del hilo principal
        archivo.writeText(texto, Charsets.UTF_8)     // crea o REEMPLAZA el archivo
    }
}
```

> **Decisión documentada: sobrescribir**
> Un borrador representa “el texto actual”, no un historial. Con `appendText` (agregar al final) cada guardado duplicaría el contenido anterior. `writeText` crea el archivo si no existe y reemplaza lo que tenía. (Agregar al final serviría para un *log* o una bitácora de eventos.)

Paso 2 · Estado: lo que se ve vs. lo que hay en disco

```
data class BorradorUiState(
    val texto: String = "",           // lo que está en el editor AHORA
    val textoGuardado: String = "",   // lo que hay en DISCO (última lectura/escritura)
    val guardando: Boolean = false,
    val mensaje: String = "Leyendo borrador...",
    val error: String? = null
) {
    val hayCambiosSinGuardar: Boolean get() = texto != textoGuardado
}
```

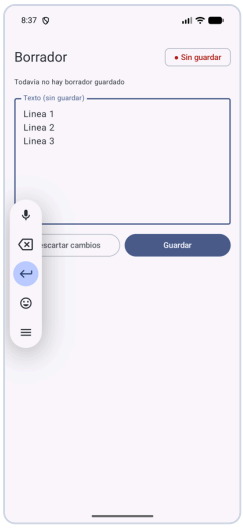
Es la misma idea del Ej 2 (“lo escrito no es lo guardado”), ahora con un disco real detrás. `hayCambiosSinGuardar` es calculada: se deduce comparando, no se guarda aparte.

Paso 3 · El ViewModel guarda y recién después confirma

```
fun onGuardar() {
    val actual = _uiState.value
    if (actual.guardando) return
    _uiState.update { it.copy(guardando = true, error = null) }
    viewModelScope.launch {
        try {
            repositorio.guardar(actual.texto)
            _uiState.update { it.copy(guardando = false, textoGuardado = actual.texto,
                                      mensaje = "Guardado en borrador.txt") }   // DESPUÉS de
escribir
        } catch (e: IOException) {
            _uiState.update { it.copy(guardando = false, error = "No se pudo guardar el
borrador") }
        }
    }
}

fun onDescartarCambios() = cargarDesdeDisco()    // desafío: relee el archivo
```

`textoGuardado` se actualiza **después** de que `guardar` terminó sin error: si la escritura falla, la pantalla sigue mostrando “sin guardar”, que es la verdad. `cargarDesdeDisco()` es la misma función que se usa al abrir: descartar = volver a leer lo que hay en disco.



Distinción visual: chip "● Sin guardar",
etiqueta "Texto (sin guardar)" y botones activos

Con el texto igual al de disco se ve "✓ Guardado" y los botones Guardar/Descartar quedan deshabilitados (no hay nada que guardar ni descartar).

Pruebas en el emulador

Prueba	Resultado
Escribir 3 líneas (con Enter) → Guardar	"✓ Guardado"; `cat files/borrador.txt` muestra las 3 líneas ✅
Cerrar completamente (`am force-stop`) y volver a abrir	"Cargado desde borrador.txt" con las 3 líneas ✅
Archivo con "Canción: ñandú / Título con acentos áéíóú", editar y Guardar	Tildes, ñ y salto de línea intactos; bytes UTF-8 (`303 263` = ó) ✅
Editar → Descartar cambios	Vuelve exactamente al texto del archivo, "✓ Guardado" ✅

Preguntas de la guía

1. ¿Qué diferencia hay entre estado del ViewModel y datos en disco?
El estado del ViewModel (`texto`) está en **memoria**: cambia en cada tecla, sobrevive a la rotación pero se pierde si muere el proceso. Los datos en disco (`borrador.txt`) son **persistentes**: sobreviven a cerrar la app y reiniciar el teléfono, pero solo cambian cuando se guarda explícitamente. Por eso pueden diferir, y esa diferencia es "cambios sin guardar".

2. ¿Qué pasa con caracteres acentuados y saltos de línea?
Se conservan siempre que se use **la misma codificación al escribir y al leer** (UTF-8 en ambos). En UTF-8 una letra acentuada ocupa 2 bytes; si se leyera con otra codificación (por ejemplo ISO-8859-1) aparecerían caracteres raros ("CanciÃ³n"). Los saltos de línea se guardan como el carácter `\n` y vuelven igual.

Resumen

1. `writeText(texto, UTF_8)` en `Dispatchers.IO`: sobrescribe el borrador.
2. `texto` (memoria) ≠ `textoGuardado` (disco) → “sin guardar”.
3. Se confirma “Guardado” solo después de escribir sin error.
4. Descartar = volver a leer el archivo.

Ejercicio 9 — Cargando, listo o falló

Consigna: modelar un estado de carga y uno de error para las operaciones de lectura/escritura. Usar `viewModelScope` para iniciar el trabajo y despachar el acceso a disco en IO. Evitar que una excepción cierre la app. Para comprobar la ruta de error, usar temporalmente un Repository de prueba que lance una excepción controlada (no alterar permisos). **Desafío:** ofrecer Reintentar si la carga falla.
Rama: `ej09-carga-error` · Archivo nuevo: `data/repository/RepositorioQueFalla.kt`

En pocas palabras

Se modelan los estados de una operación lenta: **cargando**, **listo** o **falló**. Si algo sale mal, la app no se cierra: muestra el error y ofrece reintentar. Para probarlo se usa un Repository que falla a propósito.

Importante a considerar

- **Estados con tipo sellado en vez de varias banderas** (`cargando`, `error`...): evita combinaciones imposibles como "cargando y con error a la vez".
- `try / catch` atrapa una excepción para que la app no se cierre y la convierte en un estado que la pantalla puede mostrar.
- **No atrapar cualquier excepción:** `CancellationException` es la forma normal en que Kotlin cancela corrutinas; silenciarla causa errores raros.
- **Probar errores sin romper el teléfono:** se cambia el Repository por uno falso que lanza un error. Es otra ventaja de usar una interface.
- **Reintentar** es simplemente volver a llamar a la misma función de carga.

Paso 0 · El problema de las banderas sueltas

Con `cargando: Boolean`, `error: String?` y `texto` por separado se pueden armar combinaciones absurdas: ¿qué muestra la pantalla si `cargando = true` *y* `error != null`? La teoría (diapositiva 12) propone un **tipo sellado**: cada operación está en *exactamente uno* de sus estados posibles.

Paso 1 · Dos tipos sellados: lectura y guardado

```
sealed interface EstadoCarga {
    data object Cargando : EstadoCarga
    data object Listo : EstadoCarga
    data class Error(val mensaje: String) : EstadoCarga
}

sealed interface EstadoGuardado {
    data object Inactivo : EstadoGuardado     // todavía no se intentó
    data object Guardando : EstadoGuardado
    data object Guardado : EstadoGuardado     // SOLO después de completar la escritura
    data class Error(val mensaje: String) : EstadoGuardado
}

data class BorradorUiState(
    val carga: EstadoCarga = EstadoCarga.Cargando,
    val guardado: EstadoGuardado = EstadoGuardado.Inactivo,
    val texto: String = "", val textoGuardado: String = "", val existeArchivo: Boolean = false
) {
    val puedeEditar: Boolean get() = carga == EstadoCarga.Listo
    val puedeGuardar: Boolean get() = puedeEditar && hayCambiosSinGuardar && guardado !=
EstadoGuardado.Guardando
}
```

Son dos tipos porque son dos operaciones independientes: puede fallar el guardado con la carga en `Listo`. Las propiedades calculadas (`puedeEditar`, `puedeGuardar`) dejan las reglas en el estado y no en la vista.

Paso 2 · El ViewModel: `viewModelScope` + IO + `try/catch`

```
private fun cargar() {
    _uiState.update { it.copy(carga = EstadoCarga.Cargando) }
    viewModelScope.launch {                       // inicia el trabajo
        try {
            val r = repositorio.leer()            // el Repository usa
withContext(Dispatchers.IO)
            _uiState.update { it.copy(carga = EstadoCarga.Listo, texto = ..., textoGuardado =
...) }
        } catch (e: IOException) {
            _uiState.update { it.copy(carga = EstadoCarga.Error("No se pudo leer el borrador."))
}
        }
    }
}
fun onReintentar() = cargar()                     // desafío
```

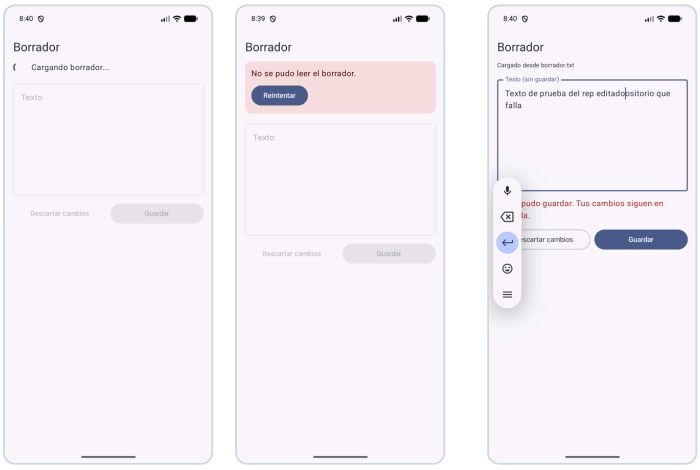
- **Quién hace qué con los hilos:** el ViewModel *inicia* con `viewModelScope.launch` (hilo principal); el Repository *despacha* el disco con `withContext(Dispatchers.IO)`. El ViewModel no necesita saber que hay un disco.
- **La excepción se captura** y se convierte en estado: la app no se cierra y el usuario ve qué pasó.
- **`onReintentar`** reutiliza la misma función de carga.

> **¿Qué excepción NO conviene capturar silenciosamente?**
> `CancellationException`. Es la forma en que Kotlin **cancela** una corrutina (por ejemplo, cuando el ViewModel se destruye y `viewModelScope` se cancela). Un `catch (e: Exception)` genérico la atraparía, la corrutina seguiría corriendo como si nada y se podría mostrar un error falso. Por eso se captura **solo** `IOException` (las fallas de disco esperables). Si alguna vez se captura `Exception`, hay que relanzar la cancelación: `catch (e: CancellationException) { throw e }`.

Paso 3 · Repository de prueba que falla (sin tocar permisos)

```
class RepositorioQueFalla(
    private var fallasDeLectura: Int = 1,        // la 1ª lectura falla, la 2ª funciona → probar Reintentar
    private val fallarEscritura: Boolean = true,
    private val demoraMs: Long = 1200            // para ver "Cargando..." en pantalla
) : BorradorRepository {
    override suspend fun leer(): ResultadoLectura {
        delay(demoraMs)
        if (fallasDeLectura > 0) { fallasDeLectura--; throw IOException("Falla de lectura simulada") }
        ...
    }
    override suspend fun guardar(texto: String) {
        delay(demoraMs)
        if (fallarEscritura) throw IOException("Falla de escritura simulada")
    }
}
// Se activa con una constante en la fábrica:
const val USAR_REPOSITORIO_QUE_FALLA = false
```

Es otra implementación del **mismo contrato**: el ViewModel y la pantalla no cambian. Esto es lo que el Ej 6 anticipaba (“la interfaz ayuda a probar”).



Cargando (editor deshabilitado) | Falló la carga → Reintentar | Falló el guardado: el texto sigue en pantalla

Pruebas en el emulador

Prueba (con `RepositorioQueFalla`)	**Resultado**
Abrir la app	"Cargando borrador..." con indicador; editor y botones deshabilitados ✅
Termina la lectura (falla)	Tarjeta roja "No se pudo leer el borrador." + **Reintentar**; la app no se cierra ✅
Reintentar	Vuelve a "Cargando..." y luego muestra el texto ✅
Editar → Guardar (escritura falla)	"Guardando..." y después "No se pudo guardar. Tus cambios siguen en pantalla."; el texto editado se conserva ✅
Con el repositorio real (`false`)	Carga el archivo, Guardar → "✓ Guardado" y el archivo cambia ✅

Preguntas de la guía

1. ¿Qué excepción no conviene capturar silenciosamente?
`CancellationException` (ver recuadro): es el mecanismo de cancelación de las corrutinas. Tampoco conviene un `catch` genérico que "trague" cualquier error sin mostrarlo ni registrarlo: el problema queda oculto. Acá cada falla se registra en Logcat *y* se muestra al usuario.

2. ¿Por qué un mensaje "Guardado" sólo corresponde después de completar la escritura?
Porque si se mostrara al *pedir* guardar y la escritura fallara, el usuario creería que sus datos están a salvo, cerraría la app y los perdería. El estado tiene que describir la verdad: `Guardando` mientras se escribe, `Guardado` solo cuando `guardar()` terminó sin excepción, `Error` si falló.

> **Resumen**
>
> 1. Tipos sellados (`EstadoCarga`, `EstadoGuardado`) en vez de banderas sueltas; `when` exhaustivo en la vista.
> 2. ViewModel inicia con `viewModelScope`; el Repository despacha el disco en IO.
> 3. Capturar `IOException`, nunca silenciar `CancellationException`.
> 4. "Guardado" solo después de escribir. Reintentar reutiliza la carga.
> 5. Las fallas se prueban con un Repository falso, no tocando el dispositivo.

Ejercicio 10 — De un texto a una lista persistente

Consigna: extender el ejercicio a varias notas. Elegir un formato de texto claro y documentado para persistirlas (JSON textual o registros con escape bien definido; no `split(";")` si el usuario puede escribir punto y coma). Implementar lectura, alta, edición y borrado a través del Repository. **Desafío:** escribir primero en un temporal y sustituir el destino sólo cuando la nueva versión esté completa.
Rama: `ej10-lista-persistente` · Archivos: `domain/repository/NotaRepository.kt` (contrato + `ArchivoNotasRepository`), `presentation/notas/NotasViewModel.kt`, `presentation/notas/PantallaNotas.kt`, `domain/validacion/ValidadorNota.kt` (vuelve del Ej 5)

> **En pocas palabras**
>
> Se pasa de un solo texto a **varias notas** guardadas en un archivo con formato **JSON**. Se puede agregar, editar y borrar, y los datos sobreviven al reinicio aunque tengan saltos de línea, comillas o punto y coma.

> **Importante a considerar**
>
> - **Por qué no separar con `;`:** si el usuario escribe un `;`, la nota se "parte". JSON sabe escapar esos caracteres (`\n`, `\"`).
> - **JSON** es un formato de texto con llaves `{}` para objetos y corchetes `[]` para listas. Android trae `org.json` para leerlo y escribirlo.
> - **Si el archivo está dañado, NO devolver lista vacía:** el próximo guardado sobrescribiría el archivo y se perderían todas las notas.
> - **Escribir en un temporal y después renombrar:** si la app se corta a mitad, el archivo original queda intacto.
> - `Mutex` hace que dos operaciones sobre el archivo no se mezclen: se ejecutan de a una.
> - **Editar = guardar con el mismo id.** Si el id ya existe, se reemplaza; si no, se agrega.

Paso 0 · ¿Por qué no `split(";")`?

Si cada nota se guardara como `titulo;contenido` en una línea, una nota con contenido `"Linea 2; con punto y coma"` se partiría en dos campos, y un salto de línea partiría la nota en dos registros. Hace falta un formato que defina cómo **escapar** los caracteres especiales.

Paso 1 · Formato documentado: `notas.json`

```
{
  "version": 1,
  "notas": [
    { "id": "24d7e63e-...", "titulo": "Parcial DA1", "contenido": "Linea 1\nLinea 2; con punto y
coma" },
    { "id": "507e3e76-...", "titulo": "Cita",        "contenido": "Dijo: \"usa JSON\" y ya" }
  ]
}
```

Decisión	Por qué
JSON (con `org.json`)	Define el escape: `\n` para salto de línea, `\"` para comillas, `\\` para la barra. El `;` no es especial. `org.json` viene incluido en Android: **sin dependencias nuevas**.
Campo vacío = `""`	Una nota vacía se representa con cadenas vacías, nunca omitiendo el campo ni con `null` (igual, el validador no deja guardarlas).
`"version": 1`	Si mañana cambia el formato, el lector sabe qué versión está leyendo.
Archivo inexistente = lista vacía	Es la primera vez que se usa la app: no es un error.
Contenido corrupto = **error**, no lista vacía	Ver Paso 3.

Paso 2 · El Repository: leer todo → modificar → escribir todo

```
class ArchivoNotasRepository(carpeta: File) : NotaRepository {
    private val mutex = Mutex()

    override suspend fun obtenerTodas() = mutex.withLock { leerArchivo() }

    override suspend fun guardar(nota: Nota) = mutex.withLock {      // alta o edición
        val notas = leerArchivo().toMutableList()
        val i = notas.indexOfFirst { it.id == nota.id }
        if (i >= 0) notas[i] = nota else notas.add(nota)
        escribirArchivo(notas)
    }

    override suspend fun eliminar(id: String) = mutex.withLock {
        escribirArchivo(leerArchivo().filterNot { it.id == id })
    }
}
```

- **El contrato es el mismo del Ej 6** (`obtenerTodas`, `guardar`, `eliminar`): cambió la implementación, de memoria a archivo.
- **`guardar` sirve para alta y edición**: si el id ya existe, reemplaza; si no, agrega.
- **`Mutex`**: cada operación es “leer, modificar, escribir”. Si dos se mezclaran (borrar mientras se guarda), una pisaría a la otra. `mutex.withLock { }` hace que se ejecuten de a una.

Paso 3 · Lector robusto: contenido incompleto

```
try {
    val raiz = JSONObject(texto)
    ...
} catch (e: JSONException) {
    throw FormatoInvalidoException("El archivo notas.json tiene un formato inválido", e)
}
class FormatoInvalidoException(...) : IOException(...)
```

> **¿Por qué no devolver una lista vacía si el archivo está dañado?**
> Porque el próximo `guardar` haría "leer (vacío) → agregar una → escribir" y **borraría todas las notas** que quedaban en el archivo. Al lanzar una excepción, el ViewModel muestra el error con *Reintentar* y el formulario queda deshabilitado: el archivo no se toca. `FormatoInvalidoException` extiende `IOException`, así que el ViewModel la maneja igual que cualquier falla de lectura (Ej 9).

Paso 4 · Desafío: escritura segura con temporal

```
private suspend fun escribirArchivo(notas: List<Nota>) = withContext(Dispatchers.IO) {
    val raiz = JSONObject().put("version", 1).put("notas", array)
    temporal.writeText(raiz.toString(2), Charsets.UTF_8)     // 1. versión nueva COMPLETA en
notas.json.tmp
    if (!temporal.renameTo(archivo))                          // 2. reemplazo atómico
        throw IOException("No se pudo reemplazar notas.json")
}
```

Si la app se corta a mitad de la escritura (batería, Android mata el proceso), solo queda un `.tmp` a medias y `notas.json` sigue intacto. Renombrar dentro de la misma carpeta es **atómico**: el archivo final es la versión vieja completa o la nueva completa, nunca una mezcla.

Paso 5 · ViewModel: un formulario para alta y edición

```
data class NotasUiState(
    val carga: EstadoCarga = EstadoCarga.Cargando,
    val notas: List<Nota> = emptyList(),
    val editandoId: String? = null,     // null = nota nueva; si no, la que se edita
    val titulo: String = "", val contenido: String = "",
    val errorTitulo: String? = null, val errorContenido: String? = null,
    val procesando: Boolean = false, val errorOperacion: String? = null
)

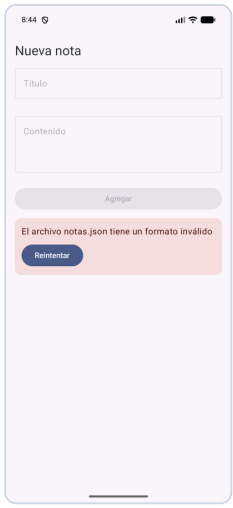
fun onEditar(nota: Nota)  // copia la nota al formulario (editandoId = nota.id)
fun onGuardar()           // valida; id = editandoId ?: UUID nuevo; repositorio.guardar(nota)
fun onEliminar(id: String)

private fun ejecutar(mensajeError: String, operacion: suspend () -> Unit) {
    // procesando = true → operacion() → recargar desde el Repository → procesando = false
    // si falla: errorOperacion = mensajeError
}
```

`ejecutar` recibe una **función suspend como parámetro** (`suspend () -> Unit`): evita repetir el mismo try/catch en guardar y eliminar. Después de cada operación se vuelve a leer del Repository: la lista mostrada es siempre la del archivo (**fuente de verdad**).



Editando una nota
(fila resaltada, “Guardar cambios”)



notas.json truncado:
error + Reintentar, archivo intacto

Pruebas en el emulador

Prueba	Resultado
Alta de 3 notas: con salto de línea y `;`, con comillas, con comas	“Notas guardadas (3)”; en el JSON: `\n` y `\"` escapados ✅
Eliminar “Cita”	Quedan 2 ✅
Editar “Compras” → agregar “y mate” → Guardar cambios	Se reemplaza la misma nota (mismo id) ✅
`am force-stop` y reabrir	Las 2 notas con saltos de línea y puntuación intactos ✅
`notas.json` truncado a 60 bytes	“El archivo notas.json tiene un formato inválido” + Reintentar; el archivo no se sobrescribe ✅
`ValidadorNotaTest`	5/5 ✅

Preguntas de la guía

1. ¿Cómo se representa una nota vacía?
Con cadenas vacías: `{"id": "...", "titulo": "", "contenido": ""}`. Los campos siempre están presentes (nunca omitidos ni `null`), así el lector no tiene casos especiales. En la práctica el validador impide guardarlas.

2. ¿Qué hace el lector si el contenido está incompleto?
`JSONObject` lanza `JSONException`; el Repository la convierte en `FormatoInvalidoException` (una `IOException`). El ViewModel muestra el error con Reintentar y bloquea el formulario. No devuelve lista vacía para no pisar el archivo en el próximo guardado.

3. ¿Qué se perdería si se reescribe mal el archivo?
Todas las notas: el archivo se reescribe entero en cada operación. Un corte a mitad de escritura dejaría un JSON truncado e ilegible. Por eso el desafío: escribir en `.tmp` y renombrar recién cuando está completo.

Resumen

1. Formato JSON documentado: escapa `;` , comillas y saltos de línea; versión; campos siempre presentes.
2. Mismo contrato `NotaRepository` , nueva implementación con archivo; `Mutex` para leer-modificar-escribir.
3. Archivo corrupto → error, nunca lista vacía.
4. Escritura segura: temporal + renombrado atómico.
5. Un formulario para alta y edición (`editandoId`).

Ejercicio 11 — Dependencias: ¿quién crea el Repository?

Consigna: revisar dónde se construyen el Repository y el ViewModel. Crear una forma sencilla de suministrar la dependencia desde la capa de aplicación (una fábrica explícita alcanza). El ViewModel debe recibir el contrato, no crear un archivo usando un Context tomado de la UI. Si se usa `AndroidViewModel`, justificar. **Desafío:** alternar entre Repository en memoria y de archivos mediante una única decisión de configuración.
Rama: `ej11-dependencias` · Archivos nuevos: `di/ContenedorApp.kt`, `data/repository/NotaRepositoryEnMemoria.kt`

En pocas palabras

Se ordena **quién construye las piezas**. En vez de que cada ViewModel cree su Repository, un **contenedor** único lo crea una vez y se lo entrega a quien lo necesite. Cambiar de "memoria" a "archivo" pasa a ser una sola línea.

Importante a considerar

- **Inyección de dependencias** = las piezas reciben lo que necesitan en vez de crearlo. Suena complicado, pero acá es solo un constructor con parámetros.
- **La clase `Application`** la crea Android antes que cualquier pantalla y vive mientras la app esté abierta: es el lugar ideal para el contenedor.
- **`by lazy`** crea el objeto la primera vez que se pide y después devuelve siempre el mismo.
- **Una sola instancia de Repository** para toda la app: si hubiera dos, cada una podría tener datos distintos.
- **Hay que registrar la Application en el Manifest** (`android:name=".MvvmLabApp"`); si no, Android usa la de por defecto.

Paso 0 · ¿Dónde se construía todo hasta ahora?

Ejercicio	Quién creaba el Repository	Problema
Ej 6	Valor por defecto en el constructor del ViewModel	El ViewModel elegía su propia implementación
Ej 7–10	La fábrica de cada ViewModel: `ArchivoNotasRepository(app.filesDir)`	Cada fábrica crea *su* Repository: con dos ViewModels habría dos instancias distintas

Inyección de dependencias (a mano)

Una pieza **no crea** lo que necesita: lo **recibe** (por constructor). Otro lugar, uno solo, se encarga de construir y conectar las piezas. Hilt o Koin automatizan esto; acá se hace "a mano" con un **contenedor**, que alcanza para una app chica.

Paso 1 · El contenedor y la única decisión de configuración

```
enum class OrigenDatos { MEMORIA, ARCHIVO }

class ContenedorApp(private val app: Application) {

    val origenDatos: OrigenDatos = OrigenDatos.ARCHIVO      // ← ÚNICA decisión (desafío)

    val notaRepository: NotaRepository by lazy {
        when (origenDatos) {
            OrigenDatos.MEMORIA -> NotaRepositoryEnMemoria()
            OrigenDatos.ARCHIVO -> ArchivoNotasRepository(app.filesDir)
        }
    }
}
```

- **`enum class`**: un tipo con un conjunto fijo de valores. El `when` es exhaustivo.
- **`by lazy { }`**: se construye la primera vez que alguien lo pide y después devuelve **siempre la misma instancia**. No se crea en cada recomposición ni en cada ViewModel.
- El tipo declarado es `NotaRepository` (el contrato): quien lo use no sabe cuál de las dos implementaciones recibió.

Paso 2 · ¿Dónde vive el contenedor? En la `Application`

```
class MvvmLabApp : Application() {
    lateinit var contenedor: ContenedorApp
        private set
    override fun onCreate() {
        super.onCreate()
        contenedor = ContenedorApp(this)
    }
}

<!-- AndroidManifest.xml -->
<application android:name=".MvvmLabApp" ...>
```

Android crea la `Application` **antes que cualquier Activity** y la mantiene mientras viva el proceso: es la "capa de aplicación". Una Activity se destruye al rotar; la Application no. `lateinit var` = "se inicializa después (en `onCreate`), pero antes de usarse".

Paso 3 · La fábrica pide, no construye

```
val Factory: ViewModelProvider.Factory = viewModelFactory {
    initializer {
        val app = checkNotNull(this[APPLICATION_KEY]) as MvvmLabApp
        NotasViewModel(app.contenedor.notaRepository)     // recibe el CONTRATO ya armado
    }
}
```

`NotasViewModel`, `PantallaNotas` y `MainActivity` no cambiaron: el ViewModel ya recibía `NotaRepository` por constructor. Lo único que cambió es **quién** lo construye.

> **¿Y `AndroidViewModel`?**
> `AndroidViewModel` es un ViewModel que recibe el `Application` (y por lo tanto un `Context`). **No se usa**: este ViewModel no necesita Context para nada; solo lo usaríamos para construir el Repository adentro, que es justamente lo que queremos evitar. Además, un ViewModel sin Context se prueba con JUnit puro (Ej 13).

Diagrama

```
MvvmLabApp (Application, 1 vez)
  └─ ContenedorApp ── origenDatos = ARCHIVO
       └─ notaRepository (by lazy, UNA instancia) : NotaRepository
            └─ NotasViewModel.Factory ──► NotasViewModel(repositorio)
                                            └─ PantallaNotas (estado ↓ / eventos ↑)
```

Pruebas en el emulador (desafío)

Configuración	Resultado
`OrigenDatos.ARCHIVO`	Muestra las 2 notas que ya estaban en `notas.json` ✅
Cambiar *una línea* a `OrigenDatos.MEMORIA`	Arranca sin notas; se agrega una → "Notas guardadas (1)" ✅
`MEMORIA` + `force-stop` y reabrir	Sin notas: la memoria se perdió con el proceso ✅
Volver a `ARCHIVO`	Las 2 notas del archivo siguen ahí (nunca se tocaron) ✅

Preguntas de la guía

1. ¿Qué se ganaría al reemplazar el Repository sin tocar el ViewModel?
Que el ViewModel (y su lógica: validaciones, estados de carga, errores) queda probado y estable aunque cambie el almacenamiento: memoria → archivo → Room → red. Se puede usar uno en memoria para desarrollar, uno falso para tests (Ej 13) y el real en la app, cambiando una sola línea.

2. ¿Qué instancia debe compartir la app?
Una sola instancia de cada Repository para toda la app. Si cada ViewModel creara el suyo: con `NotaRepositoryEnMemoria` cada uno tendría *su propia lista* (dos pantallas mostrarían datos distintos); con `ArchivoNotasRepository` cada uno tendría *su propio Mutex*, y dos escrituras simultáneas podrían pisarse. Una instancia = una fuente de verdad. Los ViewModels, en cambio, son uno por pantalla.

> **Resumen**
> 1. Las piezas reciben sus dependencias; el contenedor las construye en un solo lugar.
> 2. El contenedor vive en la `Application` (capa de aplicación, una vez por proceso).
> 3. `by lazy`: una única instancia de Repository compartida.
> 4. La fábrica pide el contrato al contenedor; el ViewModel no sabe cuál recibe.
> 5. Cambiar de origen de datos = una línea (`origenDatos`).

Anexo A · Organización del proyecto en carpetas

El código está organizado por **capas**, con la misma estructura que el repositorio de referencia *PokedexAcademy-MVVM-Clean*. Cada carpeta es un *paquete* de Kotlin (`com.example.mvvmlab.presentation.nota`, etc.). En cada rama aparecen solo las carpetas que ese ejercicio usa.

```
app/src/main/java/com/example/mvvmlab/
 ├── MainActivity.kt          # punto de entrada: aloja la UI
 ├── MvvmLabApp.kt            # Application propia (desde el Ej 11)
 ├── core/
 │    ├── model/              # tipos comunes a varias pantallas (EstadoCarga)
 │    ├── navigation/         # rutas de navegación (desde el Ej 12)
 │    ├── theme/              # tema de Compose (Color, Theme, Type)
 │    └── util/               # utilidades (Fechas)
 ├── data/                    # CÓMO se guardan los datos
 │    ├── local/              # Room: Entity, DAO, Database (Ej 14–15)
 │    ├── mapper/             # Entity ⇄ modelo de dominio
 │    └── repository/         # implementaciones de los contratos (memoria, archivos, Room)
 ├── domain/                  # QUÉ es la app: Kotlin puro, sin Android
 │    ├── model/              # Nota, Registro, ResultadoLectura
 │    ├── repository/         # contratos (interfaces) de los Repository
 │    └── validacion/         # reglas de negocio (ValidadorNota, ValidadorRegistro)
 ├── presentation/            # MVVM: una carpeta por pantalla (UiState + ViewModel + Pantalla)
 └── di/                      # quién crea las dependencias (ContenedorApp, Ej 11)

app/src/test/java/com/example/mvvmlab/
 ├── domain/validacion/       # tests de reglas
 ├── fake/                    # Repository falsos para tests
 ├── presentation/            # tests de ViewModels
 └── util/                    # MainDispatcherRule
```

Capa	Regla
`domain/`	No importa nada de Android, Compose, Room ni archivos. Define los datos (`model`), los contratos (`repository`) y las reglas (`validacion`).
`data/`	Implementa los contratos de `domain`. Es la única capa que conoce archivos, SQL o Room.
`presentation/`	ViewModels y pantallas. Solo habla con los contratos de `domain`, nunca con `data` directamente.
`di/`	Conecta todo: elige qué implementación de `data` recibe cada ViewModel.

> **¿Y los *use cases* del repositorio de referencia?**
>
> El repositorio de referencia tiene además `domain/usecase/`. Acá no se agregaron: la guía pide no crear capas “porque sí”, y en estos ejercicios cada use case solo llamaría a un método del Repository. La teoría (diapositiva 4) aclara que la capa de dominio es opcional.

Anexo B · Cómo ejecutar y probar

```
git clone https://github.com/feliforte14/MVVMlab.git
cd MVVMlab
git checkout ej15-cuaderno-de-campo        # o la rama del ejercicio
```

1. Abrir en Android Studio, Gradle sync y **Run ▶**.
2. Tests unitarios (ViewModels, validador, repositorios falsos): `./gradlew :app:testDebugUnitTest`.
3. Para probar la persistencia: cerrar la app desde Recientes y volver a abrirla; para la muerte del proceso: `adb shell am kill com.example.mvvmlab` con la app en segundo plano.
4. Ver archivos y base: *Device Explorer* (`/data/data/com.example.mvvmlab/files`) y *App Inspection → Database Inspector*.

Anexo C · Preguntas transversales (sección 18 de la guía)

Pregunta	Respuesta
1. ¿La pantalla presenta estado o también decide cómo guardarlo?	Desde el Ej 3 solo presenta: el ViewModel decide y desde el Ej 6 el Repository guarda.
2. ¿Qué clase conoce Context? ¿Lo necesita realmente?	Solo el contenedor de dependencias (en el `Application`, Ej 11) para ubicar `filesDir` y crear la base; los ViewModels no lo reciben. La View lo usa únicamente para lanzar Intents (Ej 15).
3. ¿Qué sobrevive a recomposición, rotación, muerte del proceso y reinicio?	Recomposición: `remember`. Rotación: ViewModel y `rememberSaveable`. Muerte del proceso y reinicio: solo lo escrito en disco (archivos, Room) — Ej 3, 8 y 14.
4. ¿Se accede a disco fuera del hilo principal?	Sí: los repositorios de archivo y Room trabajan en `Dispatchers.IO` / funciones `suspend` (Ej 7 en adelante).
5. ¿Un error se comunica al usuario o solo se imprime en Logcat?	Se modela en el estado (tipos sellados de carga y guardado) y se muestra en pantalla (Ej 9).
6. ¿El ViewModel se puede probar reemplazando el Repository?	Sí: recibe el contrato por constructor; los tests usan fakes y una regla para el dispatcher principal (Ej 13).
7. ¿Hay una sola fuente de verdad para cada dato?	Sí: la lista vive en el Repository y las pantallas la observan; en el integrador, la tabla de Room (Ej 12 y 15).
8. ¿Qué cambio de implementación debería ser invisible para Compose?	Pasar de memoria a archivo o a Room: cambia el Repository y el contenedor, no las pantallas (Ej 6, 10 y 14).

Anexo D · Registro breve de uso de IA (sección 19 de la guía)

La guía pide conservar, para cada bloqueo resuelto con IA: **problema, fragmento enviado, explicación recibida, cambio realizado y prueba concreta** con que se verificó. “No alcanza con anotar ‘la IA lo arregló’”.

Problema	Fragmento enviado	Explicación recibida	Cambio realizado	Prueba
Ejemplo: al rotar se perdía la lista	La pantalla con `remember { mutableStateListOf() }`	La Activity se recrea; el estado tiene que vivir en un ViewModel	Lista movida al ViewModel como `StateFlow`	Rotar el emulador: la lista sigue (Ej 3)

Buen pedido (sección 1.1): “Esta pantalla tiene estas tres variables y estas dos operaciones. Ayudame a decidir cuáles pertenecen a la UI y cuáles al ViewModel. No escribas la app entera.”

Glosario

Término	Qué significa, en simple
Activity	Una pantalla de Android. Android la crea y la destruye (por ejemplo, al rotar el teléfono). Nuestra app tiene una sola: `MainActivity`.
Composable	Función marcada con `@Composable` que *describe* un pedazo de pantalla. No devuelve nada: "emite" interfaz.
Recomposición	Cuando un dato que la pantalla usa cambia, Compose vuelve a ejecutar las funciones composables afectadas para redibujar.
Estado	Los datos que determinan cómo se ve la pantalla *ahora* (qué hay escrito, si hay un error, si está cargando).
Evento	Algo que pasó: un click, una tecla. Produce un cambio de estado.
MVVM	Forma de ordenar el código en tres partes: **View** (dibuja), **ViewModel** (decide y guarda el estado de pantalla) y **Model** (los datos y dónde se guardan).
ViewModel	Clase que guarda el estado de una pantalla y su lógica. Sobrevive a la rotación; no sobrevive a que Android cierre la app.
Repository	Clase que sabe *dónde* están los datos (memoria, archivo, base de datos) y ofrece operaciones como guardar o eliminar.
Interface (contrato)	Lista de funciones que una clase promete tener, sin decir cómo las hace. Permite cambiar la implementación sin tocar a quien la usa.
data class	Clase de Kotlin pensada para guardar datos. Trae gratis `copy`, `equals` y `toString`.
val / var	`val` no se puede reasignar; `var` sí.
null y `?`	`null` = "no hay valor". Un tipo con `?` (`String?`) puede ser null; Kotlin obliga a contemplarlo.
Lambda	Función sin nombre que se pasa como dato: `{ texto -> println(texto) }`. Se usa para avisar eventos (`onClick = { ... }`).
Corrutina	Tarea que puede esperar (leer un archivo, una demora) *sin congelar* la pantalla. Se lanza con `launch`.
suspend	Marca una función que puede "pausarse" mientras espera. Solo se llama desde una corrutina u otra función suspend.
Flow / StateFlow	Un "canal" de valores que cambian con el tiempo. Quien lo observa recibe cada valor nuevo. `StateFlow` siempre tiene un valor actual.
Fábrica (Factory)	Objeto que le explica a Android cómo construir un ViewModel que necesita parámetros.
filesDir	Carpeta privada de la app en el teléfono. Ahí se guardan archivos sin pedir permisos.

Room	Biblioteca de Android para usar una base de datos SQLite con tablas, consultas y Kotlin.